My LeetCode Submissions - @user0700N

Download PDF | Follow @TheShubham99 on GitHub | Star on GitHub | View Source Code

392 Is Subsequence (link)

Description

Given two strings `s` and `t`, return `true` *if* `s` *is a* ***subsequence*** *of* `t`*, or* `false` *otherwise*.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"abcde"` while `"aec"` is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"
Output: false
```

Constraints:

- `0 <= s.length <= 100`
- `0 <= t.length <= 10^4`
- `s` and `t` consist only of lowercase English letters.

Follow up: Suppose there are lots of incoming `s`, say s_1, s_2, ..., s_k where `k >= 10^9`, and you want to check one by one to see if `t` has its subsequence. In this scenario, how would you change your code?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isSubsequence(String s, String t) {
        if(s.isEmpty()) return true;
        int i=0,j=0;
        while(i<t.length() && j< s.length()){
            if(t.charAt(i)==s.charAt(j)){
               j++;
            }
            i++;
                if(j==s.length()) return true;

        }
        return false;

    }
}
```

28 Find the Index of the First Occurrence in a String (link)

Description

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
```

Example 2:

```
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.
```

Constraints:

- `1 <= haystack.length, needle.length <= 10^4`
- `haystack` and `needle` consist of only lowercase English characters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int strStr(String haystack, String needle) {
            if(haystack.length()<needle.length()) return -1;
            int i=0,j=0,k=0;
            while(i<haystack.length()){
                if(haystack.charAt(i)==needle.charAt(j)){
                    k=i;
                 while(j<needle.length() && i< haystack.length() && haystack.charAt(i)==n
                    i++; j++;
                }
                if(j==needle.length()) return k;
                j=0;i=k+1;
                }else{
                    j=0;
                    i++;
                }

            }
            return -1;

    }
}
```

14 Longest Common Prefix (link)

Description

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string `""`.

Example 1:

```
Input: strs = ["flower","flow","flight"]
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.
```

Constraints:

- `1 <= strs.length <= 200`
- `0 <= strs[i].length <= 200`
- `strs[i]` consists of only lowercase English letters if it is non-empty.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    StringBuilder rec(int st,StringBuilder sb,String[] strs){
        if(st==strs.length)
        return sb;
        char [] ch1 =strs[st].toCharArray();
        StringBuilder temp =new StringBuilder(sb);
        char[] arr = sb.toString().toCharArray();
                int n=Math.min(ch1.length,arr.length);
sb = new StringBuilder();
        for(int i=0;i<n;i++){
            if(ch1[i]==arr[i] )
            sb.append(ch1[i]);
            else break;
        }

       return rec(st+1,sb,strs);


    }
    public String longestCommonPrefix(String[] strs) {
        StringBuilder ans= new StringBuilder();
        ans.append(strs[0]);
      return rec(1,ans,strs).toString();



    }
}
```

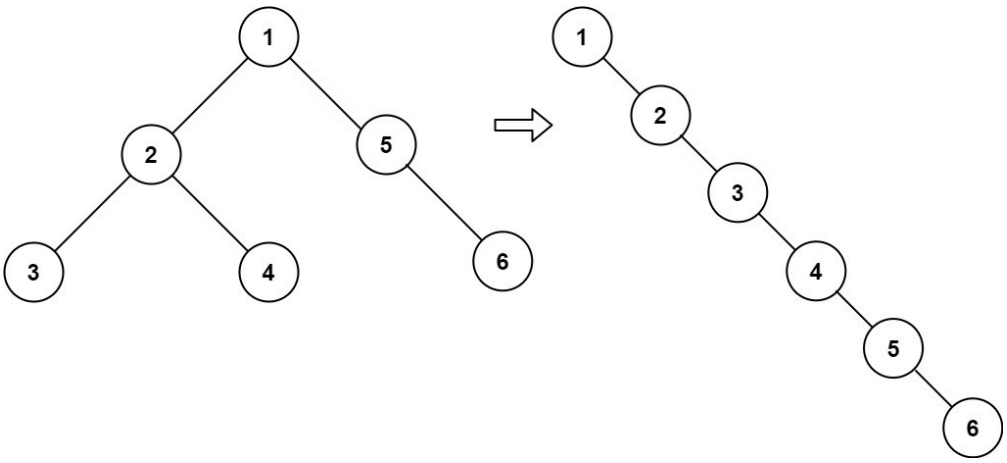
114 Flatten Binary Tree to Linked List (link)

Description

Given the `root` of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a **pre-order traversal** of the binary tree.

Example 1:




```
Input: root = [1,2,5,3,4,null,6]
Output: [1,null,2,null,3,null,4,null,5,null,6]
```

Example 2:

```
Input: root = []
Output: []
```

Example 3:

```
Input: root = [0]
Output: [0]
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-100 <= Node.val <= 100`

Follow up: Can you flatten the tree in-place (with `O(1)` extra space)?

(scroll down for solution)

Solution

Language: java

Status: Accepted

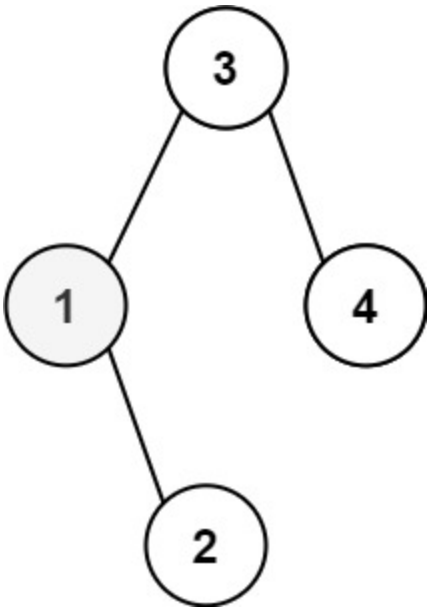
```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public void flatten(TreeNode root) {
        if(root==null)
        return ;
        if(null==root.right && null==root.left) return ;

        flatten(root.left);
         flatten(root.right);

        if(root.left!=null){
                TreeNode temp= root.right;
               root.right=root.left;
                root.left = null;
            TreeNode curr = root.right;
            while (curr.right != null) {
                curr = curr.right;
            }

            // Attach original right subtree
            curr.right = temp;

        }









    }
}
```

230 Kth Smallest Element in a BST (link)

Description

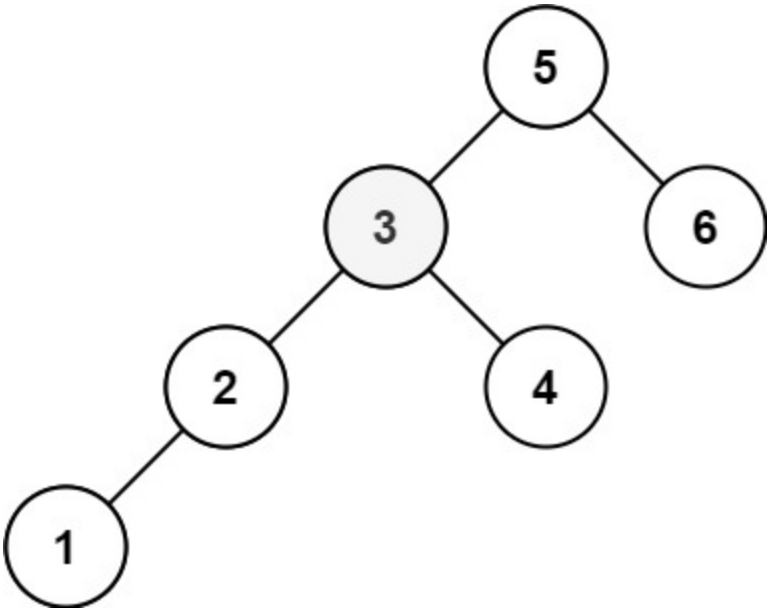
Given the `root` of a binary search tree, and an integer `k`, return *the* k^{th} *smallest value (**1-indexed**) of all the values of the nodes in the tree*.

Example 1:




```
Input: root = [3,1,4,null,2], k = 1
Output: 1
```

Example 2:



```
Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3
```

Constraints:

- The number of nodes in the tree is `n`.
- `1 <= k <= n <= 10^4`
- `0 <= Node.val <= 10^4`

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int count = 0;
    int result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;

        inorder(node.left, k);

        count++;
        if (count == k) {
            result = node.val;
            return; // stop further recursion
        }

        inorder(node.right, k);
    }
}
```

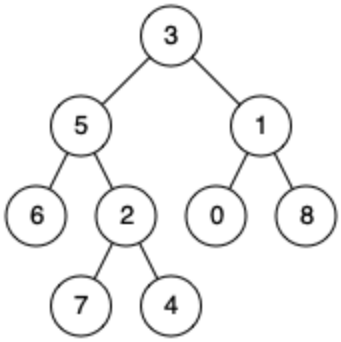
236 Lowest Common Ancestor of a Binary Tree (link)

Description

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

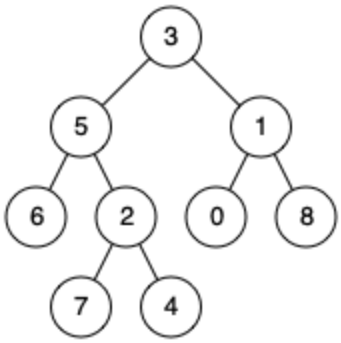
According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.
```

Example 2:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself ac
```

Example 3:

```
Input: root = [1,2], p = 1, q = 2
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[2, 10^5]`.
- `-10^9 <= Node.val <= 10^9`
- All `Node.val` are **unique**.
- `p != q`
- `p` and `q` will exist in the tree.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
       if(root==null || root.val==p.val || root.val==q.val)
            return root;
        TreeNode l= lowestCommonAncestor( root.left,  p,  q);
        TreeNode r= lowestCommonAncestor( root.right,  p,  q);


        if(l==null){
            return r;
        }else if(r==null){
            return l;
        }else if(l!=null && r!=null) {
            return root;
        }


        return null;



    }
}
```

222 Count Complete Tree Nodes (link)

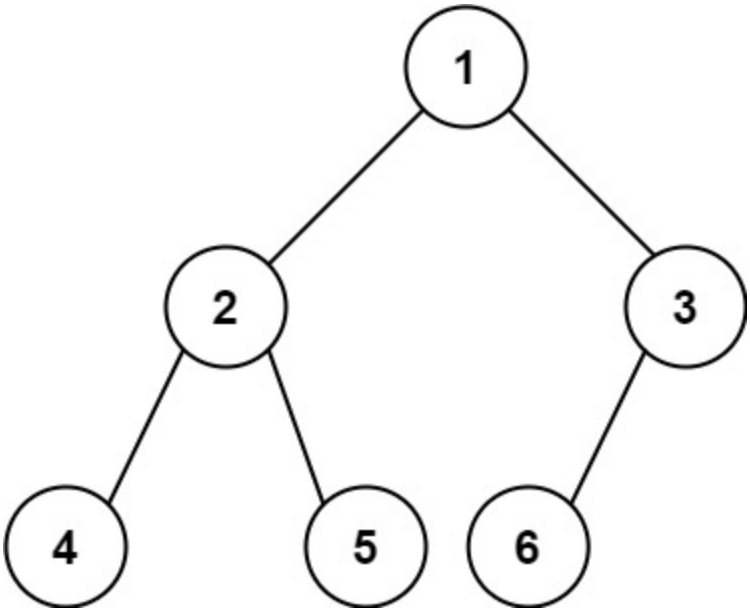
Description

Given the `root` of a **complete** binary tree, return the number of the nodes in the tree.

According to **Wikipedia**, every level, except possibly the last, is completely filled in a complete binary tree, and all nodes in the last level are as far left as possible. It can have between `1` and 2^h nodes inclusive at the last level `h`.

Design an algorithm that runs in less than `O(n)` time complexity.

Example 1:




```
Input: root = [1,2,3,4,5,6]
Output: 6
```

Example 2:

```
Input: root = []
Output: 0
```

Example 3:

```
Input: root = [1]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5 * 10^4]`.
- `0 <= Node.val <= 5 * 10^4`
- The tree is guaranteed to be **complete**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int countNodes(TreeNode root) {
        if(root==null) return 0;
        return 1+countNodes(root.left)+countNodes(root.right);

    }
}
```

129 Sum Root to Leaf Numbers (link)

Description

You are given the `root` of a binary tree containing digits from `0` to `9` only.

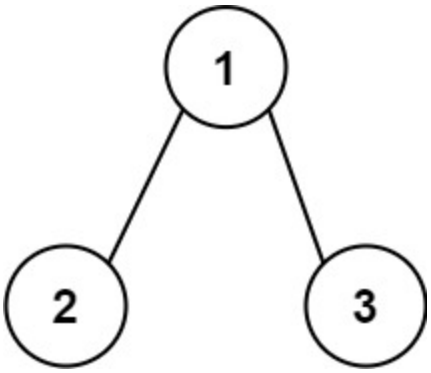
Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path `1 -> 2 -> 3` represents the number `123`.

Return *the total sum of all root-to-leaf numbers*. Test cases are generated so that the answer will fit in a **32-bit** integer.

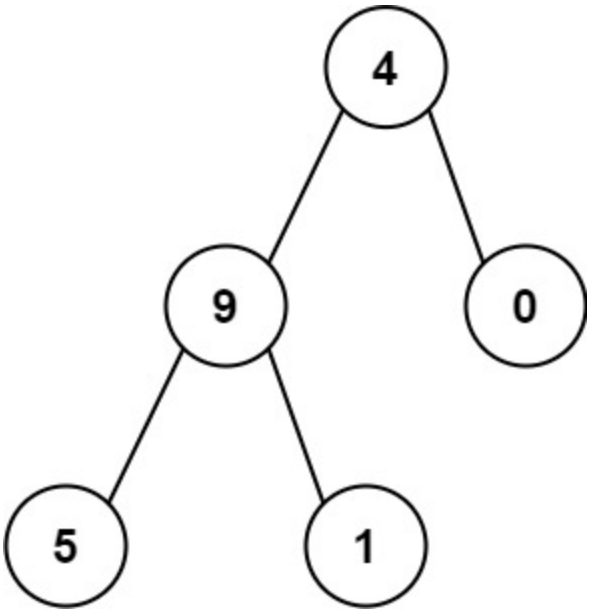
A **leaf** node is a node with no children.

Example 1:




```
Input: root = [1,2,3]
Output: 25
Explanation:
The root-to-leaf path 1->2 represents the number 12.
The root-to-leaf path 1->3 represents the number 13.
Therefore, sum = 12 + 13 = 25.
```

Example 2:




```
Input: root = [4,9,0,5,1]
Output: 1026
Explanation:
The root-to-leaf path 4->9->5 represents the number 495.
The root-to-leaf path 4->9->1 represents the number 491.
The root-to-leaf path 4->0 represents the number 40.
Therefore, sum = 495 + 491 + 40 = 1026.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `0 <= Node.val <= 9`
- The depth of the tree will not exceed `10`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int sumNumbers(TreeNode root) {
        return dfs(root, 0);
    }

    private int dfs(TreeNode node, int current) {
        if (node == null) return 0;

        // Build the number along the path
        current = current * 10 + node.val;

        // If leaf → return the number
        if (node.left == null && node.right == null) {
            return current;
        }

        // Otherwise, recurse left + right
        return dfs(node.left, current) + dfs(node.right, current);
    }
}
```

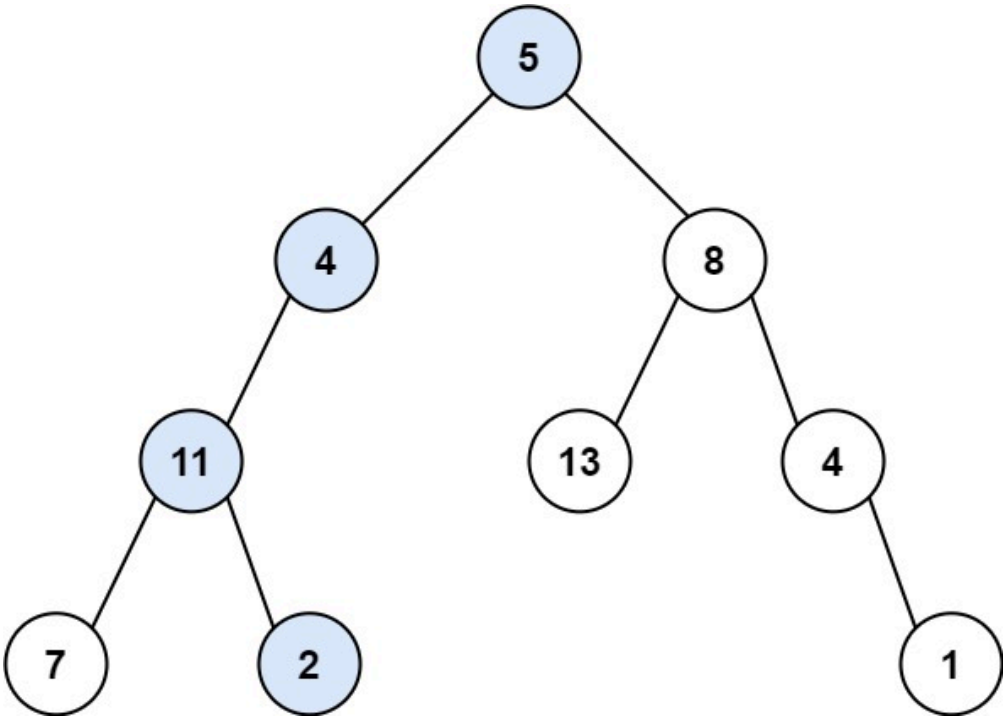
112 Path Sum (link)

Description

Given the `root` of a binary tree and an integer `targetSum`, return `true` if the tree has a **root-to-leaf** path such that adding up all the values along the path equals `targetSum`.

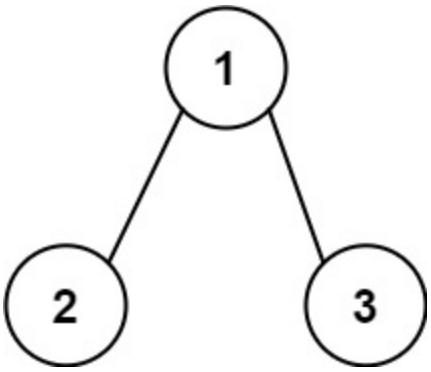
A **leaf** is a node with no children.

Example 1:




```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Output: true
Explanation: The root-to-leaf path with the target sum is shown.
```

Example 2:



```
Input: root = [1,2,3], targetSum = 5
Output: false
Explanation: There are two root-to-leaf paths in the tree:
(1 --> 2): The sum is 3.
(1 --> 3): The sum is 4.
There is no root-to-leaf path with sum = 5.
```

Example 3:

```
Input: root = [], targetSum = 0
Output: false
Explanation: Since the tree is empty, there are no root-to-leaf paths.
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5000]`.
- `-1000 <= Node.val <= 1000`
- `-1000 <= targetSum <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
          if(root==null)
          return false;
           if(targetSum==root.val && root.left==null && root.right==null)
           return true;
          int t= root==null?0:root.val;

           return hasPathSum( root.left,  targetSum-t) || hasPathSum(root.right ,  target

    }
}
```

117 Populating Next Right Pointers in Each Node II (link)

Description

Given a binary tree

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to `NULL`.

Initially, all next pointers are set to `NULL`.

Example 1:

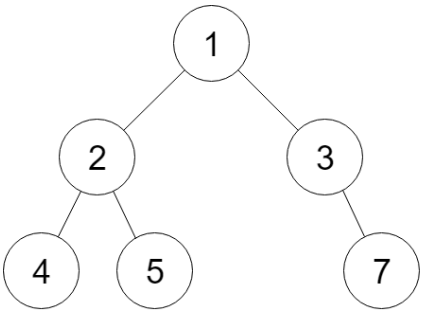


Figure A

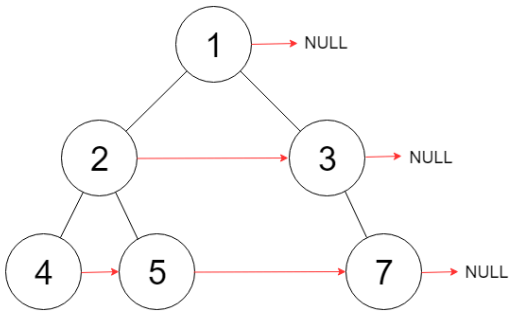


Figure B

```
Input: root = [1,2,3,4,5,null,7]
Output: [1,#,2,3,#,4,5,7,#]
Explanation: Given the above binary tree (Figure A), your function should populate each n
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 6000]`.
- `-100 <= Node.val <= 100`

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

(scroll down for solution)

Solution

Language: java

Status: Accepted

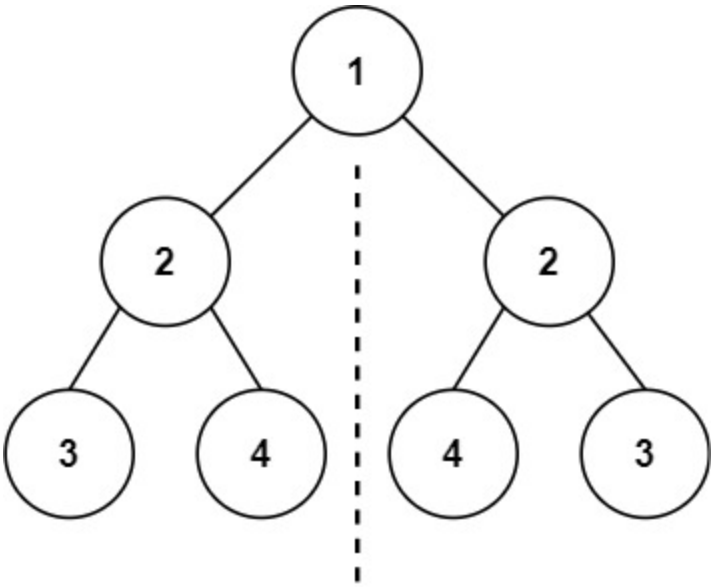
```
class Solution {
    public Node connect(Node root) {
        if (root == null) return null;

        Queue<Node> q = new LinkedList<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int size = q.size();
            Node prev = null;

            for (int i = 0; i < size; i++) {
                Node node = q.poll();

                if (prev != null) {
                    prev.next = node;   // link previous node to current
                }
                prev = node;

                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
        }

        return root;
    }
}
```

101 Symmetric Tree (link)

Description

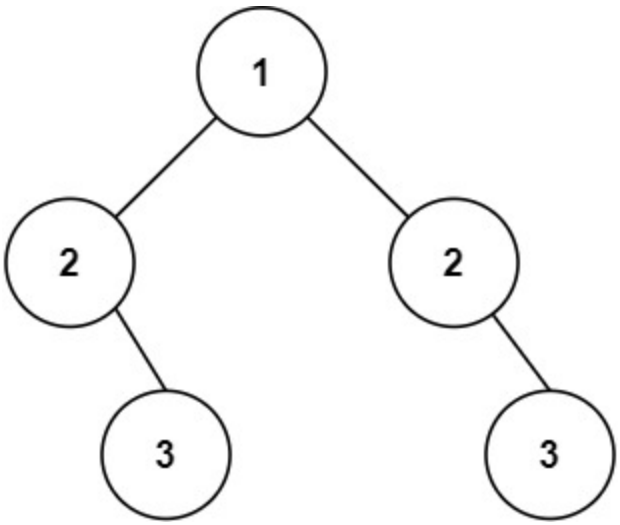
Given the `root` of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

Example 1:




```
Input: root = [1,2,2,3,4,4,3]
Output: true
```

Example 2:




```
Input: root = [1,2,2,null,3,null,3]
Output: false
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `-100 <= Node.val <= 100`

Follow up: Could you solve it both recursively and iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

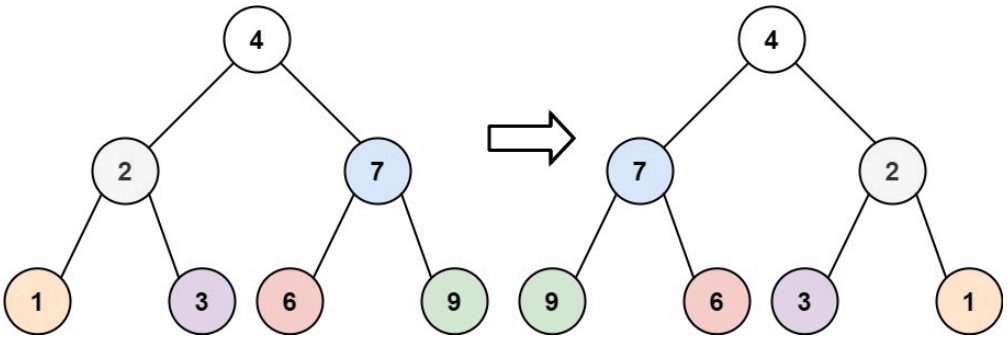
```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {

  public boolean isSymmetriutil(TreeNode p, TreeNode q){
    if(p==null && q==null) return true;
        if(p==null|| q==null) return false;


         if(p.val!=q.val) return false;
         return isSymmetriutil(p.left, q.right) && isSymmetriutil(p.right, q.left);

     }

    public boolean isSymmetric(TreeNode root) {
        if(root==null)
            return true;
      return  isSymmetriutil(root.left, root.right) && isSymmetriutil(root.right, root.le

    }
}
```

226 Invert Binary Tree (link)

Description

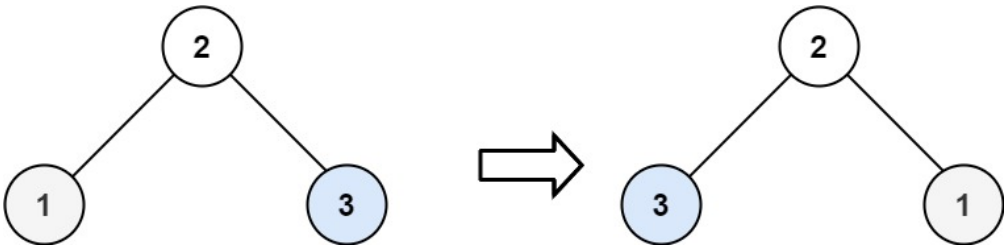
Given the `root` of a binary tree, invert the tree, and return *its root*.

Example 1:




```
Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]
```

Example 2:




```
Input: root = [2,1,3]
Output: [2,3,1]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode invertTree(TreeNode root) {
          if(root==null)
          return root;
          if(root.left==null && root.right==null)
          return root;
          TreeNode temp=root.right;
          root.right=root.left;
          root.left= temp;
           invertTree(root.left) ;
            invertTree(root.right);
            return root;



    }
}
```

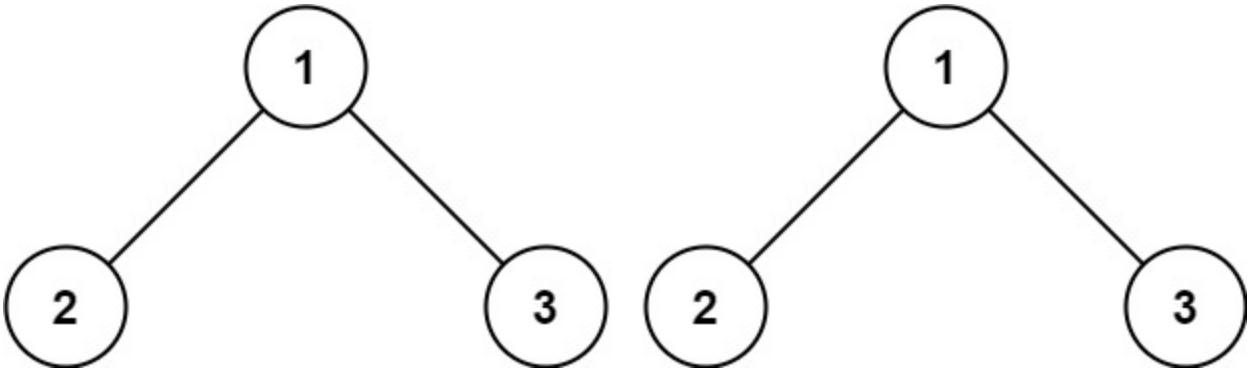
100 Same Tree (link)

Description

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

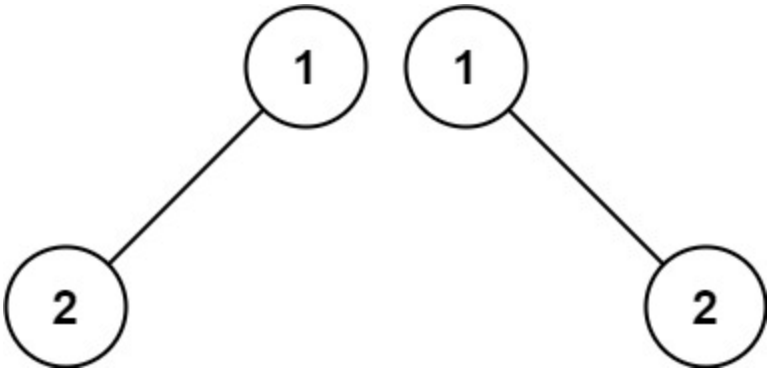
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



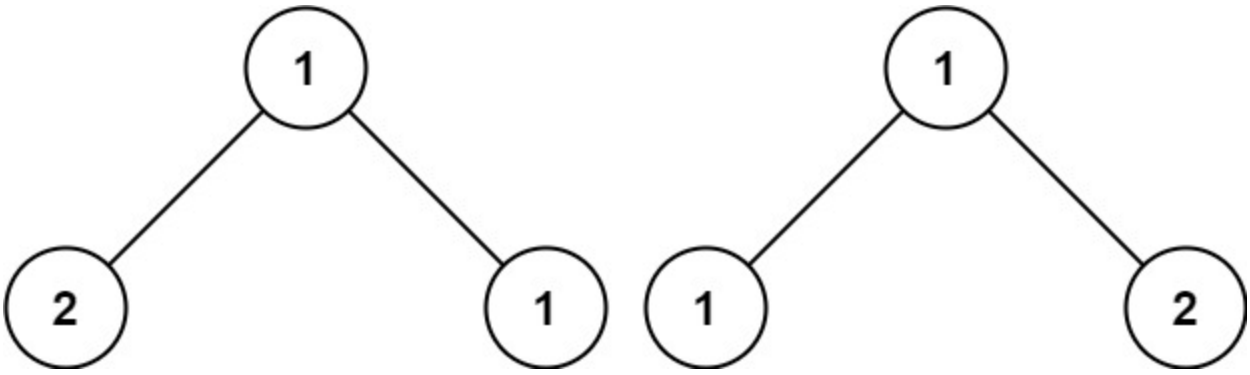

```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

Example 2:




```
Input: p = [1,2], q = [1,null,2]
Output: false
```

Example 3:




```
Input: p = [1,2,1], q = [1,1,2]
Output: false
```

Constraints:

- The number of nodes in both trees is in the range `[0, 100]`.
- `-10^4 <= Node.val <= 10^4`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if(p==null && q==null)  return true;
                if(p==null || q==null)  return false;
                        if(p.val!=q.val)  return false;

       return isSameTree(p.left,q.left) &&  isSameTree(p.right,q.right) ;


    }
}
```

237 Delete Node in a Linked List (link)

Description

There is a singly-linked list `head` and we want to delete a node `node` in it.

You are given the node to be deleted `node`. You will **not be given access** to the first node of `head`.

All the values of the linked list are **unique**, and it is guaranteed that the given node `node` is not the last node in the linked list.

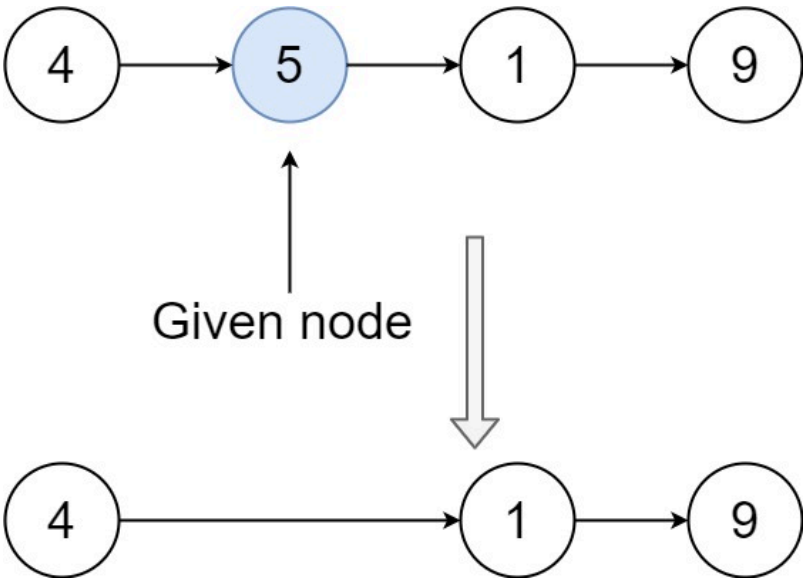
Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before `node` should be in the same order.
- All the values after `node` should be in the same order.

Custom testing:

- For the input, you should provide the entire linked list `head` and the node to be given `node`. `node` should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

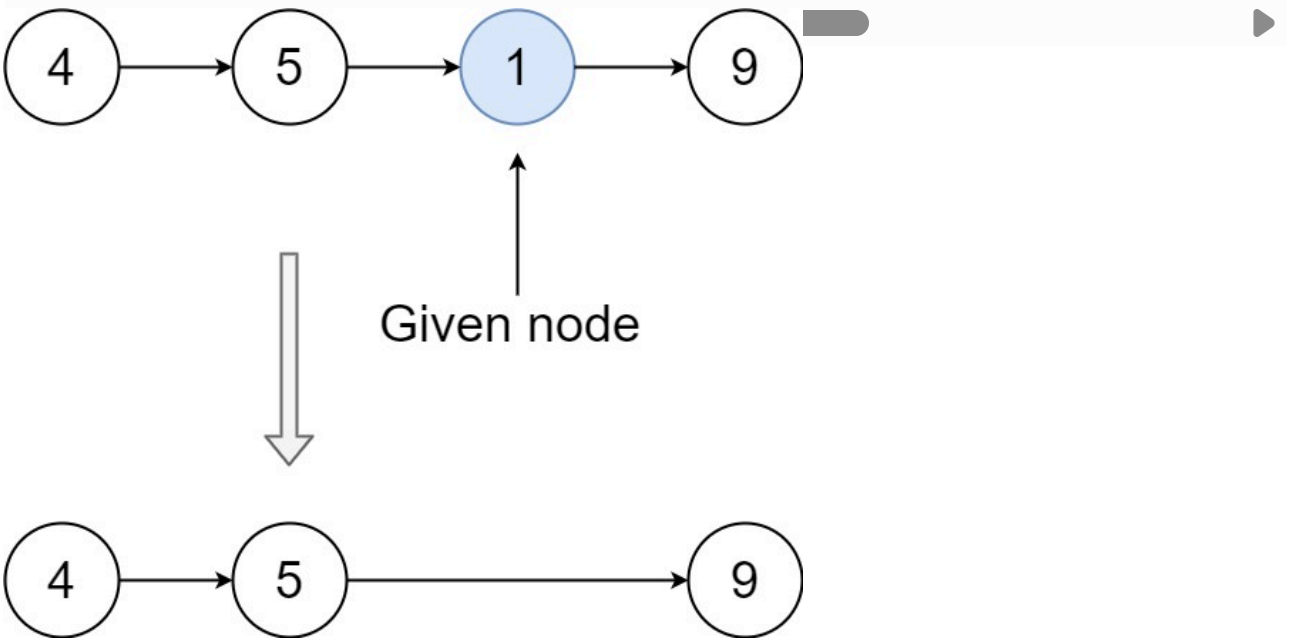
Example 1:




```
Input: head = [4,5,1,9], node = 5
Output: [4,1,9]
```

```
Explanation: You are given the second node with value 5, the linked list should become 4
```

Example 2:




```
Input: head = [4,5,1,9], node = 1
Output: [4,5,9]
Explanation: You are given the third node with value 1, the linked list should become 4
```

Constraints:

- The number of the nodes in the given list is in the range `[2, 1000]`.
- `-1000 <= Node.val <= 1000`
- The value of each node in the list is **unique**.
- The `node` to be deleted is **in the list** and is **not a tail** node.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {
    public void deleteNode(ListNode node) {
        if(node==null || node.next==null)
        return ;
         node.val = node.next.val;       // copy next node’s value
        node.next = node.next.next;

    }
}
```

146 LRU Cache (link)

Description

Design a data structure that follows the constraints of a **Least Recently Used (LRU) cache**.

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with **positive** size `capacity`.
- `int get(int key)` Return the value of the `key` if the key exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the `key` exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, **evict** the least recently used key.

The functions `get` and `put` must each run in `O(1)` average time complexity.

Example 1:

```
Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1);    // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2);    // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1);    // return -1 (not found)
lRUCache.get(3);    // return 3
lRUCache.get(4);    // return 4
```

Constraints:

- `1 <= capacity <= 3000`
- `0 <= key <= 10^4`
- `0 <= value <= 10^5`
- At most `2 * 10^5` calls will be made to `get` and `put`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class LRUCache {
    LinkedHashMap<Integer,Integer> mp;
    int capacity;

    public LRUCache(int capacity) {
        this.capacity=capacity;
               mp = new LinkedHashMap<>(capacity, 0.75f, true);

    }

    public int get(int key) {
        if(mp.get(key)==null)
        return -1;
        int r= mp.get(key);

        return r;
    }

    public void put(int key, int value) {
        mp.put(key, value);
        if(mp.size() > capacity){
            Map.Entry<Integer, Integer> firstEntry = mp.entrySet().iterator().next();
            mp.remove(firstEntry.getKey());
        }

    }
}



/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
```

1272 Invalid Transactions (link)

Description

A transaction is possibly invalid if:

- the amount exceeds `$1000`, or;
- if it occurs within (and including) `60` minutes of another transaction with the **same name** in a **different city**.

You are given an array of strings `transaction` where `transactions[i]` consists of comma-separated values representing the name, time (in minutes), amount, and city of the transaction.

Return a list of `transactions` that are possibly invalid. You may return the answer in **any order**.

Example 1:

```
Input: transactions = ["alice,20,800,mtv","alice,50,100,beijing"]
Output: ["alice,20,800,mtv","alice,50,100,beijing"]
Explanation: The first transaction is invalid because the second transaction occurs withi
```

Example 2:

```
Input: transactions = ["alice,20,800,mtv","alice,50,1200,mtv"]
Output: ["alice,50,1200,mtv"]
```

Example 3:

```
Input: transactions = ["alice,20,800,mtv","bob,50,1200,mtv"]
Output: ["bob,50,1200,mtv"]
```

Constraints:

- `transactions.length <= 1000`
- Each `transactions[i]` takes the form `"{name},{time},{amount},{city}"`
- Each `{name}` and `{city}` consist of lowercase English letters, and have lengths between `1` and `10`.
- Each `{time}` consist of digits, and represent an integer between `0` and `1000`.
- Each `{amount}` consist of digits, and represent an integer between `0` and `2000`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;

class Solution {
    static class Tx {
        String name, city;
        int time, amount;
        String raw;

        Tx(String s) {
            String[] parts = s.split(",");
            name = parts[0];
            time = Integer.parseInt(parts[1]);
            amount = Integer.parseInt(parts[2]);
            city = parts[3];
            raw = s;
        }
    }

    public List<String> invalidTransactions(String[] transactions) {
        int n = transactions.length;
        Tx[] txs = new Tx[n];
        for (int i = 0; i < n; i++) {
            txs[i] = new Tx(transactions[i]);
        }

        boolean[] invalid = new boolean[n];

        // Rule 1: amount > 1000
        for (int i = 0; i < n; i++) {
            if (txs[i].amount > 1000) {
                invalid[i] = true;
            }
        }

        // Rule 2: within 60 min, same name, diff city
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (txs[i].name.equals(txs[j].name) &&
                    Math.abs(txs[i].time - txs[j].time) <= 60 &&
                    !txs[i].city.equals(txs[j].city)) {
                    invalid[i] = true;
                    invalid[j] = true;
                }
            }
        }

        List<String> ans = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (invalid[i]) {
                ans.add(txs[i].raw);
            }
        }
        return ans;
```

```
        }
    }
```

2691 Count Vowel Strings in Ranges (link)

Description

You are given a **0-indexed** array of strings `words` and a 2D array of integers `queries`.

Each query `queries[i] = [l_i, r_i]` asks us to find the number of strings present at the indices ranging from `l_i` to `r_i` (both **inclusive**) of `words` that start and end with a vowel.

Return *an array* `ans` *of size* `queries.length`*, where* `ans[i]` *is the answer to the* `i`th *query*.

Note that the vowel letters are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`.

Example 1:

```
Input: words = ["aba","bcb","ece","aa","e"], queries = [[0,2],[1,4],[1,1]]
Output: [2,3,0]
Explanation: The strings starting and ending with a vowel are "aba", "ece", "aa" and "e"
The answer to the query [0,2] is 2 (strings "aba" and "ece").
to query [1,4] is 3 (strings "ece", "aa", "e").
to query [1,1] is 0.
We return [2,3,0].
```

Example 2:

```
Input: words = ["a","e","i"], queries = [[0,2],[0,1],[2,2]]
Output: [3,2,1]
Explanation: Every string satisfies the conditions, so we return [3,2,1].
```

Constraints:

- `1 <= words.length <= 10^5`
- `1 <= words[i].length <= 40`
- `words[i]` consists only of lowercase English letters.
- `sum(words[i].length) <= 3 * 10^5`
- `1 <= queries.length <= 10^5`
- `0 <= l_i <= r_i < words.length`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private boolean isVowel(char c) {
        return c=='a' || c=='e' || c=='i' || c=='o' || c=='u';
    }

    public int[] vowelStrings(String[] words, int[][] queries) {
        int n = words.length;
        int[] prefix = new int[n+1];  // prefix sum array

        for (int i = 0; i < n; i++) {
            String s = words[i];
            int len = s.length();
            if (isVowel(s.charAt(0)) && isVowel(s.charAt(len-1))) {
                prefix[i+1] = prefix[i] + 1;
            } else {
                prefix[i+1] = prefix[i];
            }
        }

        int[] ans = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int l = queries[i][0];
            int r = queries[i][1];
            ans[i] = prefix[r+1] - prefix[l];
        }

        return ans;
    }
}
```

1306 Minimum Absolute Difference (link)

Description

Given an array of **distinct** integers `arr`, find all pairs of elements with the minimum absolute difference of any two elements.

Return a list of pairs in ascending order(with respect to pairs), each pair `[a, b]` follows

- `a, b` are from `arr`
- `a < b`
- `b - a` equals to the minimum absolute difference of any two elements in `arr`

Example 1:

```
Input: arr = [4,2,1,3]
Output: [[1,2],[2,3],[3,4]]
Explanation: The minimum absolute difference is 1. List all pairs with difference equal t
```

Example 2:

```
Input: arr = [1,3,6,10,15]
Output: [[1,3]]
```

Example 3:

```
Input: arr = [3,8,-10,23,19,-4,-14,27]
Output: [[-14,-10],[19,23],[23,27]]
```

Constraints:

- `2 <= arr.length <= 10^5`
- `-10^6 <= arr[i] <= 10^6`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> minimumAbsDifference(int[] arr) {
        Arrays.sort(arr);
        List<List<Integer>> ans= new ArrayList<>();

        int min=arr[1]-arr[0];
        for(int i=1;i<arr.length-1;i++){
            if(arr[i+1]-arr[i]<min)
            min=arr[i+1]-arr[i];
        }
           for(int i=0;i<arr.length-1;i++){
            if(arr[i+1]-arr[i]==min)
            ans.add(List.of(arr[i],arr[i+1]));
        }
        return ans;

    }
}
```

724 Find Pivot Index (link)

Description

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-1000 <= nums[i] <= 1000`

Note: This question is the same as 1991: https://leetcode.com/problems/find-the-middle-index-in-array/

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int num : nums) {
            total += num;
        }

        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
            int rightSum = total - leftSum - nums[i];
            if (leftSum == rightSum) {
                return i;
            }
            leftSum += nums[i];
        }

        return -1;
    }
}
```

33 Search in Rotated Sorted Array (link)

Description

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

Constraints:

- `1 <= nums.length <= 5000`
- `-10^4 <= nums[i] <= 10^4`
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- `-10^4 <= target <= 10^4`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
      int   util(int st ,int end,int target , int[] num){
          if(end<st)
              return -1;
          int mid=st+(end-st)/2;
          if(num[mid]==target)
              return mid;
          else if(num[st]<=num[mid])
          {
              if(target>=num[st] && target< num[mid])
                   return    util(st ,mid-1,target , num);
              else{
                                  return    util(mid+1 ,end,target , num);

              }

          }  else if(num[end]>=num[mid]){
              if(target > num[mid] && target <= num[end])
                                  return    util(mid+1, end,target , num);
                  else{
                                                          return    util(st ,mid-1,targe

                  }


          }
                     return -1;

      }

    public int search(int[] nums, int target) {
       return util(0 ,nums.length-1,target , nums);

    }
}
```

387 First Unique Character in a String (link)

Description

Given a string `s`, find the **first** non-repeating character in it and return its index. If it **does not** exist, return `-1`.

Example 1:

Input: s = "leetcode"

Output: 0

Explanation:

The character `'l'` at index 0 is the first character that does not occur at any other index.

Example 2:

Input: s = "loveleetcode"

Output: 2

Example 3:

Input: s = "aabb"

Output: -1

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

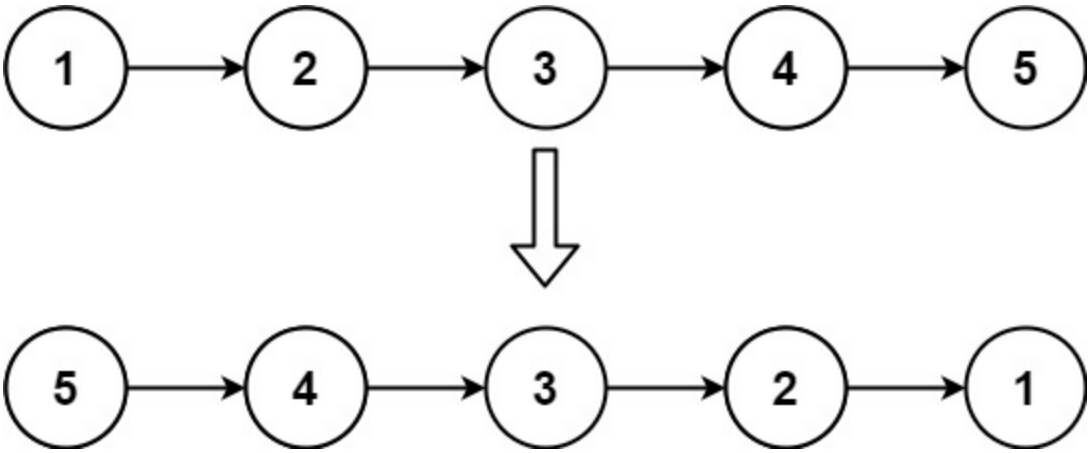
```
class Solution {
    public int firstUniqChar(String s) {
        Map<Character,Integer> st=new  HashMap<>();
        int i=0,j=0;
        while(i < s.length()){
            st.put(s.charAt(i),st.getOrDefault(s.charAt(i),0)+1);
            i++;
        }

               while(j < s.length()){
                   if(st.get(s.charAt(j))==1)
                       return j;
                   else{
                       j++;
                   }

               }
        return -1;


    }
}
```

206 Reverse Linked List (link)

Description

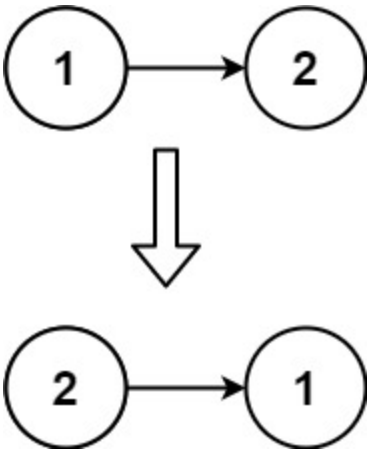
Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]
```

Example 2:




```
Input: head = [1,2]
Output: [2,1]
```

Example 3:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in the list is the range `[0, 5000]`.
- `-5000 <= Node.val <= 5000`

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode previous=null;
                ListNode next=null;

        ListNode current=head;
        while(current!=null){
            next=current.next;
            current.next=previous;
        previous= current;
            current =next;
        }
        return previous;


    }
}
```

347 Top K Frequent Elements (link)

Description

Given an integer array `nums` and an integer `k`, return *the* `k` *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than `O(n log n)`, where n is the array's size.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Pair{
    int key;
    int value;
    Pair(int key ,int val){
       this.key=key;
       this.value=value;
    }
}
class Solution {
    public int[] topKFrequent(int[] nums, int k) {


        Map<Integer,Integer> mp=new HashMap<>();
        for( int n:nums){
            mp.put(n,mp.getOrDefault(n,0)+1);
        }
        List<Map.Entry<Integer,Integer>> list=new ArrayList<>(mp.entrySet());
        Collections.sort(list,(a,b)->b.getValue()-a.getValue());
        int [] ans=new int [k];
        int i=0;
        for(Map.Entry<Integer,Integer> m:list){
            if(k>0){
               ans[i++]= m.getKey();
                k--;
            }else break;


        }
        return ans;




    }
}
```

2581 Divide Players Into Teams of Equal Skill (link)

Description

You are given a positive integer array `skill` of **even** length `n` where `skill[i]` denotes the skill of the `i`th player. Divide the players into `n / 2` teams of size `2` such that the total skill of each team is **equal**.

The **chemistry** of a team is equal to the **product** of the skills of the players on that team.

Return *the sum of the **chemistry** of all the teams, or return* `-1` *if there is no way to divide the players into teams such that the total skill of each team is equal.*

Example 1:

```
Input: skill = [3,2,5,1,3,4]
Output: 22
Explanation:
Divide the players into the following teams: (1, 5), (2, 4), (3, 3), where each team has
The sum of the chemistry of all the teams is: 1 * 5 + 2 * 4 + 3 * 3 = 5 + 8 + 9 = 22.
```

Example 2:

```
Input: skill = [3,4]
Output: 12
Explanation:
The two players form a team with a total skill of 7.
The chemistry of the team is 3 * 4 = 12.
```

Example 3:

```
Input: skill = [1,1,2,3]
Output: -1
Explanation:
There is no way to divide the players into teams such that the total skill of each team i
```

Constraints:

- `2 <= skill.length <= 10`^{5}
- `skill.length` is even.
- `1 <= skill[i] <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public long dividePlayers(int[] skill) {
        long ans=0;
        Arrays.sort(skill);
        int i=0,j=skill.length-1;
        ans=skill[i]*skill[j];
        int sm=skill[i++]+skill[j--];

        while(i<j){
            if(sm!=skill[i]+skill[j])
            return -1;
            else{
             ans+=skill[i++]*skill[j--];
            }

        }
        return ans;

    }
}
```

442 Find All Duplicates in an Array (link)

Description

Given an integer array `nums` of length `n` where all the integers of `nums` are in the range `[1, n]` and each integer appears **at most twice**, return *an array of all the integers that appears **twice***.

You must write an algorithm that runs in `O(n)` time and uses only *constant* auxiliary space, excluding the space needed to store the output

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1,2]
Output: [1]
```

Example 3:

```
Input: nums = [1]
Output: []
```

Constraints:

- `n == nums.length`
- `1 <= n <= 10^5`
- `1 <= nums[i] <= n`
- Each element in `nums` appears **once** or **twice**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findDuplicates(int[] nums) {
         List<Integer> ans=new ArrayList<>();
         for(int n:nums){
            if(nums[Math.abs(n)-1]<0)
            ans.add(Math.abs(n));
            nums[Math.abs(n)-1]=-nums[Math.abs(n)-1];

         }
         return ans;

    }
}
```

2470 Removing Stars From a String (link)

Description

You are given a string `s`, which contains stars `*`.

In one operation, you can:

- Choose a star in `s`.
- Remove the closest **non-star** character to its **left**, as well as remove the star itself.

Return *the string after **all** stars have been removed*.

Note:

- The input will be generated such that the operation is always possible.
- It can be shown that the resulting string will always be unique.

Example 1:

```
Input: s = "leet**cod*e"
Output: "lecoe"
Explanation: Performing the removals from left to right:
- The closest character to the 1st star is 't' in "leet**cod*e". s becomes "lee*cod*e".
- The closest character to the 2nd star is 'e' in "lee*cod*e". s becomes "lecod*e".
- The closest character to the 3rd star is 'd' in "lecod*e". s becomes "lecoe".
There are no more stars, so we return "lecoe".
```

Example 2:

```
Input: s = "erase*****"
Output: ""
Explanation: The entire string is removed, so we return an empty string.
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of lowercase English letters and stars `*`.
- The operation above can be performed on `s`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String removeStars(String s) {
        char [] st=s.toCharArray();
         StringBuilder sb=new StringBuilder();
        int i=0;
        while(i<s.length()){
            if(st[i]!='*'){
                sb.append(st[i]);
                i++;
            }else{
               sb.deleteCharAt(sb.length()-1);
                i++;
            }
        }


        return sb.toString();

    }
}
```

238 Product of Array Except Self (link)

Description

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in `O(n)` time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```

Constraints:

- `2 <= nums.length <= 10^5`
- `-30 <= nums[i] <= 30`
- The input is generated such that `answer[i]` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in `O(1)` extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int zero=0,index=0;
        int ans[]= new int[nums.length];
        int p=1;
        for(int i=0;i<nums.length;i++){
            if(nums[i]!=0){
                p*=nums[i];

            }else{
                zero++;
                index=i;
            }
        }
        if(zero>1)
        return ans;
        if(zero==1){
        ans[index]=p;
        return ans;
        }
       for(int i=0;i<nums.length;i++){
        ans[i]=p/nums[i];
       }
       return ans;



    }
}
```

1610 XOR Operation in an Array (link)

Description

You are given an integer `n` and an integer `start`.

Define an array `nums` where `nums[i] = start + 2 * i` (**0-indexed**) and `n == nums.length`.

Return *the bitwise XOR of all elements of* `nums`.

Example 1:

```
Input: n = 5, start = 0
Output: 8
Explanation: Array nums is equal to [0, 2, 4, 6, 8] where (0 ^ 2 ^ 4 ^ 6 ^ 8) = 8.
Where "^" corresponds to bitwise XOR operator.
```

Example 2:

```
Input: n = 4, start = 3
Output: 8
Explanation: Array nums is equal to [3, 5, 7, 9] where (3 ^ 5 ^ 7 ^ 9) = 8.
```

Constraints:

- `1 <= n <= 1000`
- `0 <= start <= 1000`
- `n == nums.length`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int xorOperation(int n, int start) {
        int i=0;
        int sum=0;
        while(i<n)
        {
            sum^=start+2*i;
            i++;
        }
        return sum;

    }
}
```

1741 Sort Array by Increasing Frequency (link)

Description

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Return the *sorted array*.

Example 1:

```
Input: nums = [1,1,2,2,2,3]
Output: [3,1,1,2,2,2]
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency
```

Example 2:

```
Input: nums = [2,3,1,3,2]
Output: [1,3,3,2,2]
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing orc
```

Example 3:

```
Input: nums = [-1,1,-6,4,5,-6,1,4,1]
Output: [5,-1,4,4,-6,-6,1,1,1]
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;
class Solution {
    public int[] frequencySort(int[] nums) {
        HashMap<Integer,Integer> mp= new HashMap<>();
        for(int n:nums){
            mp.put(n,mp.getOrDefault(n,0)+1);
        }
      List<Map.Entry<Integer, Integer>> list = new ArrayList<>(mp.entrySet());
      Collections.sort(list, (a, b) -> b.getKey()-a.getKey());
      Collections.sort(list, (a, b) -> a.getValue()-b.getValue());
     int [] ans=new int [nums.length];
     int j=0;
      for(Map.Entry<Integer, Integer> m:list){
      int c=m.getValue();
      int key=m.getKey();
      while(c>0){
        ans[j]=key;
        c--;
        j++;
      }
      }
      return ans;


    }
}
```

200 Number of Islands (link)

Description

Given an `m x n` 2D binary grid `grid` which represents a map of `'1'`s (land) and `'0'`s (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is `'0'` or `'1'`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int row[]={-1,1,0,0};
    int col[]={0,0,-1,1};
   void  dfs(int i ,int j, char[][] mt,boolean [][] visited){
        for(int k=0;k<4;k++){
            int dx=i+row[k];
            int dy=j+col[k];
            if(dx>=0 && dy>=0 && dx < mt.length && dy<mt[0].length &&  mt[dx][dy]=='1' &&
                visited[dx][dy]=true;
                dfs(dx,dy,mt,visited);
            }
        }

    }
    public int numIslands(char[][] grid) {
        int ans=0;
        int n=grid.length;
        int m=grid[0].length;
        boolean [][] visited= new boolean[n][m];

        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                if(grid[i][j]=='1' && !visited[i][j]){
                    visited[i][j]=true;
                    ans++;
                    dfs(i,j,grid,visited);
                }

            }
        }
        return ans;

    }
}
```

2426 Maximum Profit From Trading Stocks (link)

Description

You are given two **0-indexed** integer arrays of the same length `present` and `future` where `present[i]` is the current price of the i^{th} stock and `future[i]` is the price of the i^{th} stock a year in the future. You may buy each stock at most **once**. You are also given an integer `budget` representing the amount of money you currently have.

Return *the maximum amount of profit you can make.*

Example 1:

```
Input: present = [5,4,6,2,3], future = [8,5,4,3,5], budget = 10
Output: 6
Explanation: One possible way to maximize your profit is to:
Buy the 0th, 3rd, and 4th stocks for a total of 5 + 2 + 3 = 10.
Next year, sell all three stocks for a total of 8 + 3 + 5 = 16.
The profit you made is 16 - 10 = 6.
It can be shown that the maximum profit you can make is 6.
```

Example 2:

```
Input: present = [2,2,5], future = [3,4,10], budget = 6
Output: 5
Explanation: The only possible way to maximize your profit is to:
Buy the 2nd stock, and make a profit of 10 - 5 = 5.
It can be shown that the maximum profit you can make is 5.
```

Example 3:

```
Input: present = [3,3,12], future = [0,3,15], budget = 10
Output: 0
Explanation: One possible way to maximize your profit is to:
Buy the 1st stock, and make a profit of 3 - 3 = 0.
It can be shown that the maximum profit you can make is 0.
```

Constraints:

- `n == present.length == future.length`
- `1 <= n <= 1000`
- `0 <= present[i], future[i] <= 100`
- `0 <= budget <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maximumProfit(int[] present, int[] future, int budget) {
        int dp [][]= new int[future.length+1][budget+1];
        int n=future.length;
        int m=budget;
        for( int [] a: dp)
        Arrays.fill(a,-1);
        for(int i=0;i<=n;i++){
            for(int j=0;j<=m;j++){
                if(i==0){
                dp[i][j]=0;

               } else{
                    if(present[i-1]>j){
                        dp[i][j]=dp[i-1][j];

                    }else{
                        dp[i][j]=Math.max(dp[i-1][j],future[i-1]-present[i-1]+dp[i-1][j-p

                    }

                }
            }
        }
        return dp[n][m];

    }
}
```

1781 Check If Two String Arrays are Equivalent (link)

Description

Given two string arrays `word1` and `word2`, return `true` *if the two arrays* ***represent*** *the same string, and* `false` *otherwise.*

A string is **represented** by an array if the array elements concatenated **in order** forms the string.

Example 1:

```
Input: word1 = ["ab", "c"], word2 = ["a", "bc"]
Output: true
Explanation:
word1 represents string "ab" + "c" -> "abc"
word2 represents string "a" + "bc" -> "abc"
The strings are the same, so return true.
```

Example 2:

```
Input: word1 = ["a", "cb"], word2 = ["ab", "c"]
Output: false
```

Example 3:

```
Input: word1  = ["abc", "d", "defg"], word2 = ["abcddefg"]
Output: true
```

Constraints:

- `1 <= word1.length, word2.length <= 10^3`
- `1 <= word1[i].length, word2[i].length <= 10^3`
- `1 <= sum(word1[i].length), sum(word2[i].length) <= 10^3`
- `word1[i]` and `word2[i]` consist of lowercase letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean arrayStringsAreEqual(String[] word1, String[] word2) {
        StringBuilder sb = new StringBuilder();
        for(String s:word1)
            {
                sb.append(s);
            }
            StringBuilder sb1 = new StringBuilder();
        for(String s1:word2)
            {
                sb1.append(s1);
            }
        return sb.toString().equals(sb1.toString());



    }
}
```

1635 Number of Good Pairs (link)

Description

Given an array of integers `nums`, return *the number of **good pairs***.

A pair `(i, j)` is called *good* if `nums[i] == nums[j]` and `i` < `j`.

Example 1:

```
Input: nums = [1,2,3,1,1,3]
Output: 4
Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.
```

Example 2:

```
Input: nums = [1,1,1,1]
Output: 6
Explanation: Each pair in the array are good.
```

Example 3:

```
Input: nums = [1,2,3]
Output: 0
```

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```java
import java.util.*;

class Solution {
    public int numIdenticalPairs(int[] nums) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int num : nums) {
            freq.put(num, freq.getOrDefault(num, 0) + 1);
        }

        int count = 0;
        for (int f : freq.values()) {
            count += f * (f - 1) / 2;
        }
        return count;
    }
}
```

1700 Minimum Time to Make Rope Colorful (link)

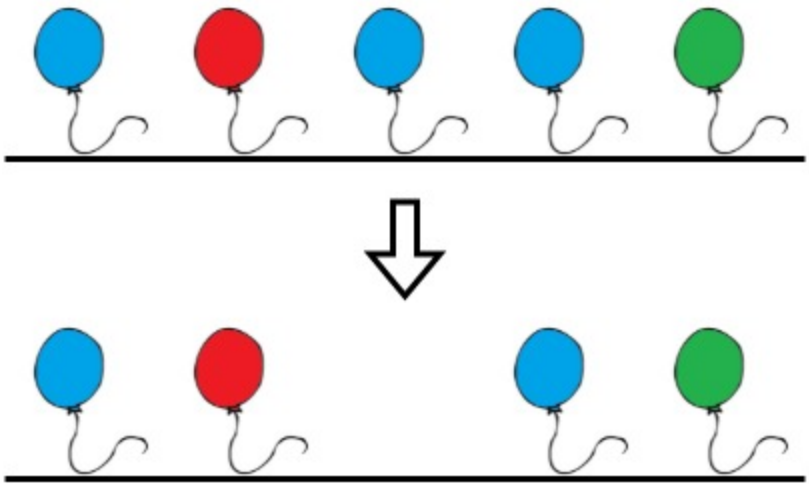
Description

Alice has `n` balloons arranged on a rope. You are given a **0-indexed** string `colors` where `colors[i]` is the color of the `i`th balloon.

Alice wants the rope to be **colorful**. She does not want **two consecutive balloons** to be of the same color, so she asks Bob for help. Bob can remove some balloons from the rope to make it **colorful**. You are given a **0-indexed** integer array `neededTime` where `neededTime[i]` is the time (in seconds) that Bob needs to remove the `i`th balloon from the rope.

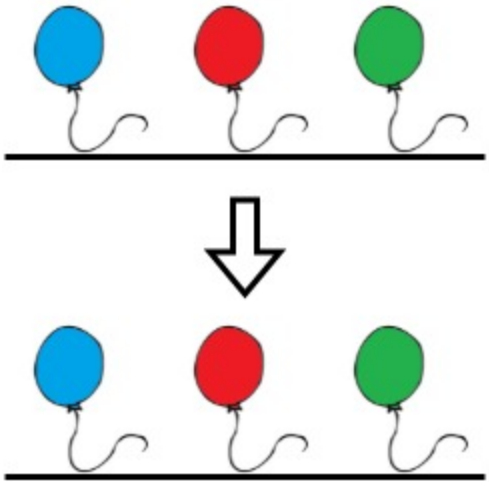
Return *the **minimum time** Bob needs to make the rope **colorful***.

Example 1:



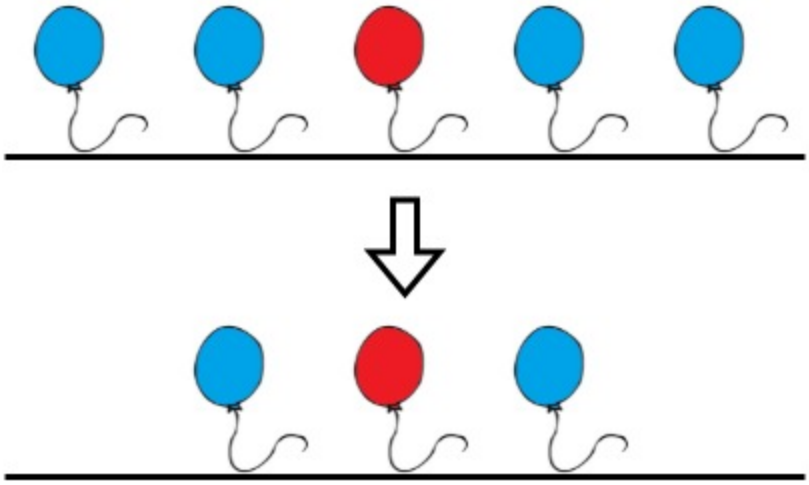
```
Input: colors = "abaac", neededTime = [1,2,3,4,5]
Output: 3
Explanation: In the above image, 'a' is blue, 'b' is red, and 'c' is green.
Bob can remove the blue balloon at index 2. This takes 3 seconds.
There are no longer two consecutive balloons of the same color. Total time = 3.
```

Example 2:



```
Input: colors = "abc", neededTime = [1,2,3]
Output: 0
Explanation: The rope is already colorful. Bob does not need to remove any balloons from
```

Example 3:



```
Input: colors = "aabaa", neededTime = [1,2,3,4,1]
Output: 2
Explanation: Bob will remove the balloons at indices 0 and 4. Each balloons takes 1 secor
There are no longer two consecutive balloons of the same color. Total time = 1 + 1 = 2.
```

Constraints:

- `n == colors.length == neededTime.length`
- `1 <= n <= 10^5`
- `1 <= neededTime[i] <= 10^4`
- `colors` contains only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minCost(String colors, int[] neededTime) {
        int n = colors.length();
        int ans = 0;

        for (int i = 1; i < n; i++) {
            if (colors.charAt(i) == colors.charAt(i - 1)) {
                // Remove the smaller cost balloon
                ans += Math.min(neededTime[i], neededTime[i - 1]);

                // Keep the larger cost as "survivor" by updating neededTime[i]
                neededTime[i] = Math.max(neededTime[i], neededTime[i - 1]);
            }
        }

        return ans;
    }
}
```

647 Palindromic Substrings (link)

Description

Given a string `s`, return *the number of **palindromic substrings** in it*.

A string is a **palindrome** when it reads the same backward as forward.

A **substring** is a contiguous sequence of characters within the string.

Example 1:

```
Input: s = "abc"
Output: 3
Explanation: Three palindromic strings: "a", "b", "c".
```

Example 2:

```
Input: s = "aaa"
Output: 6
Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countSubstrings(String s) {
        int n = s.length();
        int count = 0;

        for(int i = 0; i < n; i++){
            count += expand(s, i, i);
            count += expand(s, i, i + 1);
        }
        return count;
    }

    private int expand(String s, int left, int right){
        int c = 0;
        while(left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)){
            c++;
            left--;
            right++;
        }
        return c;
    }
}
```

1170 Shortest Common Supersequence (link)

Description

Given two strings `str1` and `str2`, return *the shortest string that has both* `str1` *and* `str2` *as* ***subsequences***. If there are multiple valid strings, return **any** of them.

A string `s` is a **subsequence** of string `t` if deleting some number of characters from `t` (possibly `0`) results in the string `s`.

Example 1:

```
Input: str1 = "abac", str2 = "cab"
Output: "cabac"
Explanation:
str1 = "abac" is a subsequence of "cabac" because we can delete the first "c".
str2 = "cab" is a subsequence of "cabac" because we can delete the last "ac".
The answer provided is the shortest such string that satisfies these properties.
```

Example 2:

```
Input: str1 = "aaaaaaaa", str2 = "aaaaaaaa"
Output: "aaaaaaaa"
```

Constraints:

- `1 <= str1.length, str2.length <= 1000`
- `str1` and `str2` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;
class Solution {

    public String shortestCommonSupersequence(String s1, String s2) {
        int m=s1.length(),n=s2.length();
        int dp[][]= new int [m+1][n+1];
        for(int i=0;i<=m;i++)
        {
            for(int j=0;j<=n;j++){
                if( i==0 || j==0)
                    dp[i][j]=0;
                    else if(s1.charAt(i-1)==s2.charAt(j-1)){
                        dp[i][j]=1+dp[i-1][j-1];

                    }else{
                        dp[i][j]=Math.max(dp[i-1][j],dp[i][j-1]);
                    }
            }
        }
            StringBuilder sb= new StringBuilder();
            while( m>0 && n>0){
                if(s1.charAt(m - 1) == s2.charAt(n - 1)){
                    sb.append(s1.charAt(m-1));
                    m--;n--;
                }else if(dp[m-1][n] > dp[m][n-1]){
                                sb.append(s1.charAt(m-1));

                    m--;
                }else{
                                sb.append(s2.charAt(n-1));

                    n--;
                }
            }

    while(m > 0) {
        sb.append(s1.charAt(m-1));
        m--;
    }
    while(n > 0) {
        sb.append(s2.charAt(n-1));
        n--;
    }

    // Reverse because we built it backwards
    return sb.reverse().toString();




        }
```

```
}
```

402 Remove K Digits (link)

Description

Given string num representing a non-negative integer `num`, and an integer `k`, return *the smallest possible integer after removing* `k` *digits from* `num`.

Example 1:

```
Input: num = "1432219", k = 3
Output: "1219"
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the
```

Example 2:

```
Input: num = "10200", k = 1
Output: "200"
Explanation: Remove the leading 1 and the number is 200. Note that the output must not co
```

Example 3:

```
Input: num = "10", k = 2
Output: "0"
Explanation: Remove all the digits from the number and it is left with nothing which is 0
```

Constraints:

- `1 <= k <= num.length <= 10^5`
- `num` consists of only digits.
- `num` does not have any leading zeros except for the zero itself.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String removeKdigits(String num, int k) {
        // Edge case
        if (num == null || num.length() == 0 || k >= num.length()) {
            return "0";
        }

        // Using StringBuilder as a stack
        StringBuilder stack = new StringBuilder();

        for (int i = 0; i < num.length(); i++) {
            char current = num.charAt(i);

            // Keep popping while:
            // 1) k > 0
            // 2) stack is not empty
            // 3) top of stack > current digit
            while (k > 0 && stack.length() > 0 && stack.charAt(stack.length() - 1) > curr
                stack.deleteCharAt(stack.length() - 1);
                k--;
            }
            // push current
            stack.append(current);
        }

        // If k is still > 0, remove from the end
        while (k > 0 && stack.length() > 0) {
            stack.deleteCharAt(stack.length() - 1);
            k--;
        }

        // Remove leading zeros
        while (stack.length() > 0 && stack.charAt(0) == '0') {
            stack.deleteCharAt(0);
        }

        // If empty, return "0"
        if (stack.length() == 0) {
            return "0";
        }

        return stack.toString();
    }
}
```

75 Sort Colors (link)

Description

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either `0`, `1`, or `2`.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void sortColors(int[] nums) {
        int a=0,b=0,c=0;
        for(int n:nums){
            if(n==0)
                a++;
            else if(n==1)
                b++;
            else
                c++;
        }
        int i=0;
        while(a>0){
            nums[i++]=0;
            a--;
        }
        while(b>0){
            nums[i++]=1;
            b--;
        }
        while(c>0){
            nums[i++]=2;
            c--;
        }


    }
}
```

42 Trapping Rain Water (link)

Description

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:


```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0
```

Example 2:

```
Input: height = [4,2,0,3,2,5]
Output: 9
```

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 10^4`
- `0 <= height[i] <= 10^5`

(scroll down for solution)

Solution

Language: java

Status: Accepted

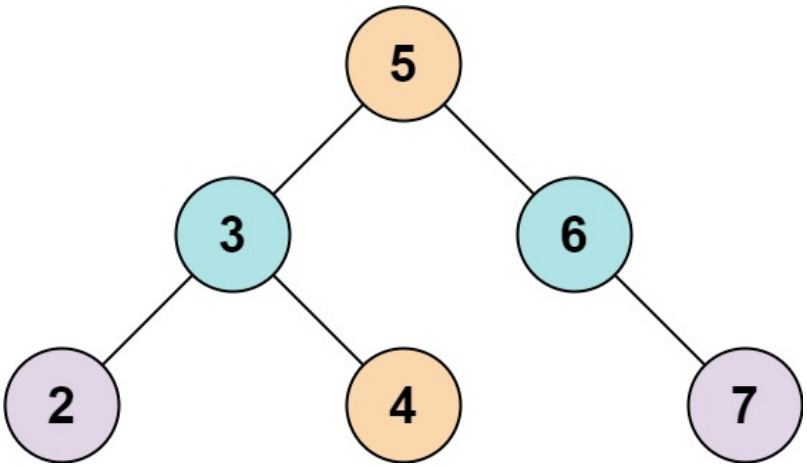
```
class Solution {
    public int trap(int[] height) {
        int ans=0;
        int l=height.length;
        int lmax=height[0];

        int rmax=height[l-1];
            int i=1, j=l-2;
        List<Integer> la= new ArrayList<>();
        List<Integer> r= new ArrayList<>();
        la.add(lmax);
        r.add(rmax);


        while(i<l && j>=0){
           la.add(Math.max(lmax,height[i]));
            if(height[i]>lmax){
                lmax=height[i];
            }
                        r.add( Math.max(rmax,height[j]));
             if(height[j]>rmax){
                rmax=height[j];
            }
                                i++;
                                j--;
        }
            Collections.reverse(r);
           for (int k = 0; k < l; k++) {
            ans += Math.min(la.get(k), r.get(k)) - height[k];
        }
        return ans;
    }
}
```

653 Two Sum IV - Input is a BST (link)

Description

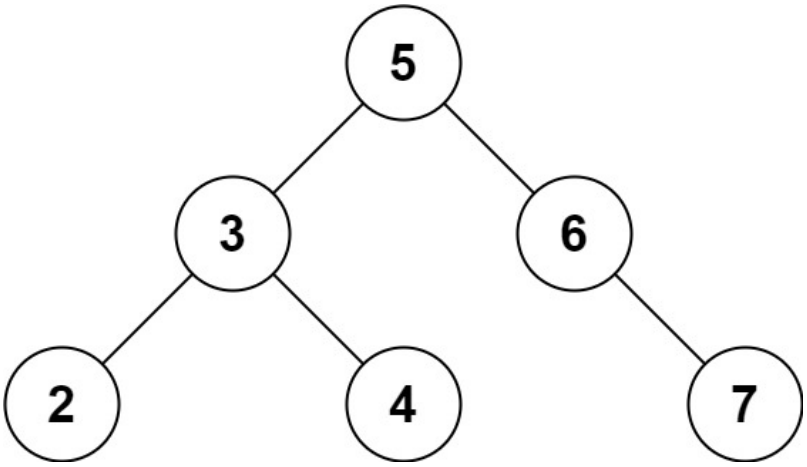
Given the `root` of a binary search tree and an integer `k`, return `true` *if there exist two elements in the BST such that their sum is equal to* `k`, *or* `false` *otherwise*.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], k = 9
Output: true
```

Example 2:




```
Input: root = [5,3,6,2,4,null,7], k = 28
Output: false
```

Constraints:

- The number of nodes in the tree is in the range `[1, 10^4]`.
- `-10^4 <= Node.val <= 10^4`
- `root` is guaranteed to be a **valid** binary search tree.
- `-10^5 <= k <= 10^5`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    List<Integer> ans= new ArrayList<>();
    void util(TreeNode root){
        if(root==null)
            return ;
        util(root.left);
        ans.add(root.val);
                util(root.right);



    }
    public boolean findTarget(TreeNode root, int k) {
        util(root);
        Collections.sort(ans);
        int i=0,j=ans.size()-1;
        while(i<j){
            if(ans.get(i)+ans.get(j)==k)
                return true;
            else if(ans.get(i)+ans.get(j)<k)
                i++;
            else
                j--;

        }
        return false;

    }
}
```

328 Odd Even Linked List (link)

Description

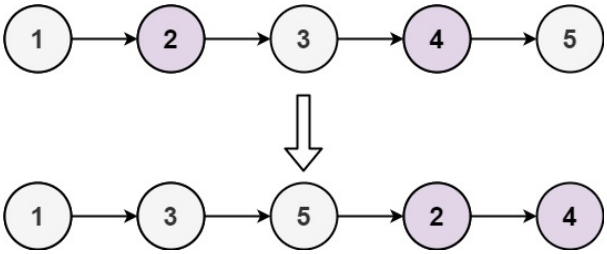
Given the `head` of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*.

The **first** node is considered **odd**, and the **second** node is **even**, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

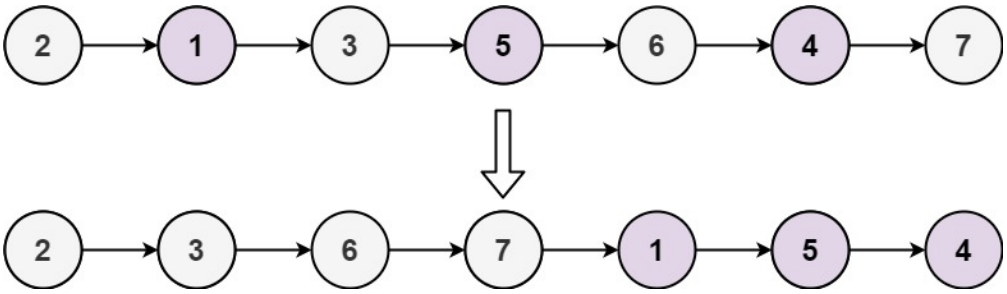
You must solve the problem in `O(1)` extra space complexity and `O(n)` time complexity.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]
```

Example 2:




```
Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]
```

Constraints:

- The number of nodes in the linked list is in the range `[0, 10^4]`.
- `-10^6 <= Node.val <= 10^6`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode oddEvenList(ListNode head) {
     if (head == null || head.next == null)
            return head;

        ListNode odd = head;
        ListNode even = head.next;
        ListNode evenHead = even; // to connect at the end

        while (even != null && even.next != null) {
            odd.next = even.next;
            odd = odd.next;

            even.next = odd.next;
            even = even.next;
        }

        odd.next = evenHead; // connect end of odd to start of even
        return head;
    }
}
```

142 Linked List Cycle II (link)

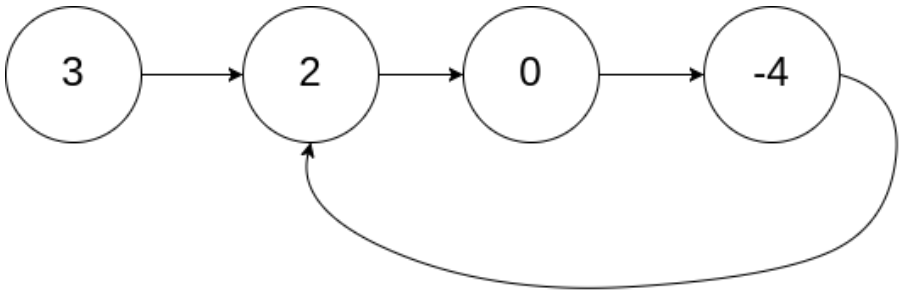
Description

Given the `head` of a linked list, return *the node where the cycle begins. If there is no cycle, return* `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that** `pos` **is not passed as a parameter**.

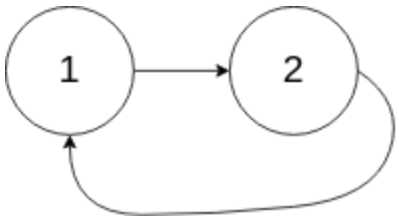
Do not modify the linked list.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: tail connects to node index 1
Explanation: There is a cycle in the linked list, where tail connects to the second node
```

Example 2:




```
Input: head = [1,2], pos = 0
Output: tail connects to node index 0
Explanation: There is a cycle in the linked list, where tail connects to the first node.
```

Example 3:


```
Input: head = [1], pos = -1
Output: no cycle
Explanation: There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range `[0, 10^4]`.
- `-10^5 <= Node.val <= 10^5`
- `pos` is `-1` or a **valid index** in the linked-list.

Follow up: Can you solve it using `O(1)` (i.e. constant) memory?

(scroll down for solution)

Solution

Language: java

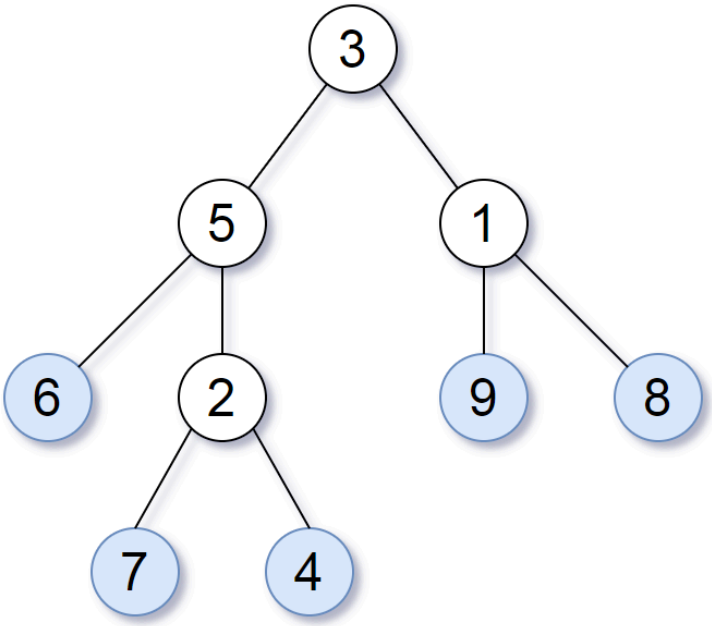
Status: Accepted

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
import java.util.*;
public class Solution {
    boolean util(ListNode head){
           if(head==null || head.next==null)
           return false;
        ListNode fast=head;
        ListNode slow=head;
        while(fast!=null && fast.next!=null){
            fast=fast.next.next;
            slow=slow.next;
            if(fast==slow)
                return  true;

        }
        return false;

    }
    public ListNode detectCycle(ListNode head) {
        int i=0;

Map<ListNode, Integer> s = new HashMap<>();
        ListNode hea=head;


        if(util(head)){
            while(hea!=null){
            if(s.containsKey(hea))
                 return hea;
            else{
                s.put(hea,i);
                i++;
                hea=hea.next;
            }
            }
        }
        return null;

    }
}
```

904 Leaf-Similar Trees (link)

Description

Consider all the leaves of a binary tree, from left to right order, the values of those leaves form a **leaf value sequence**.

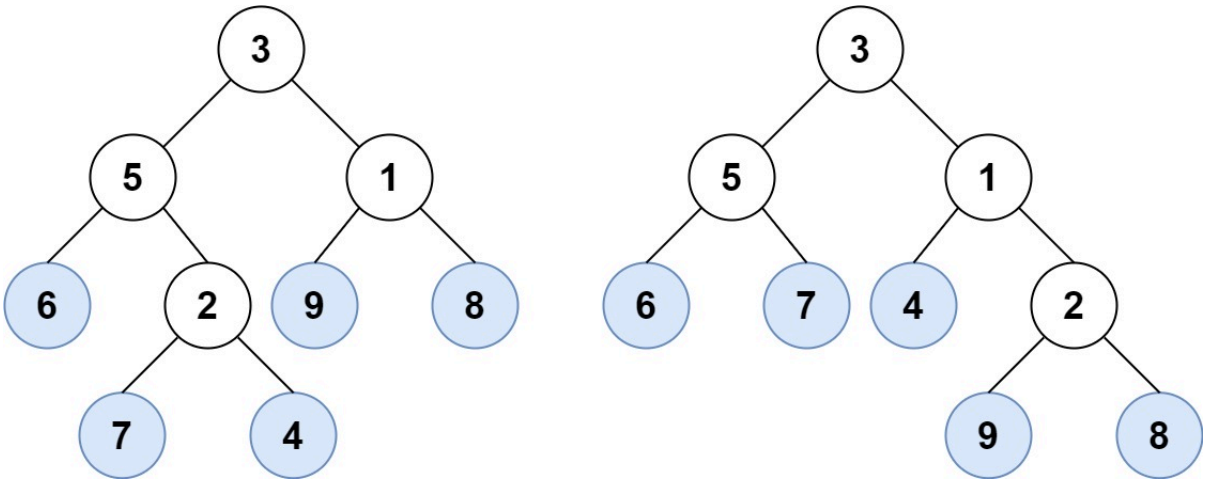


For example, in the given tree above, the leaf value sequence is `(6, 7, 4, 9, 8)`.

Two binary trees are considered *leaf-similar* if their leaf value sequence is the same.

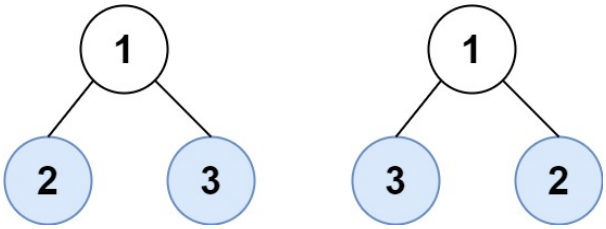
Return `true` if and only if the two given trees with head nodes `root1` and `root2` are leaf-similar.

Example 1:



```
Input: root1 = [3,5,1,6,2,9,8,null,null,7,4], root2 = [3,5,1,6,7,4,2,null,null,null,null,
Output: true
```

Example 2:




```
Input: root1 = [1,2,3], root2 = [1,3,2]
Output: false
```

Constraints:

- The number of nodes in each tree will be in the range `[1, 200]`.
- Both of the given trees will have values in the range `[0, 200]`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
     List<Integer> ans1= new ArrayList<Integer>();
        List<Integer> ans2= new ArrayList<Integer>();
  void  util(TreeNode root){
        if(root==null)
            return;



        if(root.left==null &&  root.right==null){
                         ans1.add(root.val);
            return;}



    util(root.left);
                 util(root.right);


   }
      void  util2(TreeNode root){
        if(root==null)
            return;

          if(root.left==null &&  root.right==null){
                         ans2.add(root.val);
              return;

        }
              util2(root.left);
   util2(root.right);


      }


    public boolean leafSimilar(TreeNode root1, TreeNode root2) {

         util(root1);
         util2(root2);
        return ans1.equals(ans2);
```

```
        }
    }
```

861 Flipping an Image (link)

Description

Given an `n x n` binary matrix `image`, flip the image **horizontally**, then invert it, and return *the resulting image*.

To flip an image horizontally means that each row of the image is reversed.

- For example, flipping `[1,1,0]` horizontally results in `[0,1,1]`.

To invert an image means that each `0` is replaced by `1`, and each `1` is replaced by `0`.

- For example, inverting `[0,1,1]` results in `[1,0,0]`.

Example 1:

```
Input: image = [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
```

Example 2:

```
Input: image = [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
```

Constraints:

- `n == image.length`
- `n == image[i].length`
- `1 <= n <= 20`
- `images[i][j]` is either `0` or `1`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] flipAndInvertImage(int[][] a) {
        for(int k=0;k<a.length;k++){
    int i=0,j=a[k].length-1;
        while(i<j){
            int t=a[k][i];
            a[k][i]=a[k][j];
            a[k][j]=t;
            i++;
            j--;
        }
        }



        for(int i=0;i<a.length;i++){
            for(int j=0;j<a[0].length;j++){
                if(a[i][j]==0)
                    a[i][j]=1;
                else
                    a[i][j]=0;
            }
        }

            return a;

    }
}
```

34 Find First and Last Position of Element in Sorted Array (link)

Description

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

Example 3:

```
Input: nums = [], target = 0
Output: [-1,-1]
```

Constraints:

- `0 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`
- `nums` is a non-decreasing array.
- `-10^9 <= target <= 10^9`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int [] util(int[] nums, int target,int[] ans,int i,int j){
        if(i>j || j>= nums.length)
        return ans;
         int mid=i+(j-i)/2;
        if(nums[mid]<target)
       return  util(nums,target,ans,mid+1,j);
        else    if(nums[mid]>target)
        return  util(nums,target,ans,i,mid-1);
        else{
            int k=mid,l=mid;
            while(k>=0 && nums[k]==target ){
                k--;
            }
            while(  l<nums.length && nums[l]==target ){
                l++;
            }
            ans[0]=k+1;
             ans[1]=l-1;
             return ans;


        }



    }
    public int[] searchRange(int[] nums, int target) {
        int [] ans =new int[2];
        ans[0]=-1;
        ans[1]=-1;
        int i=0,j=nums.length-1;
      return  util(nums,target,ans,i,j);


    }
}
```

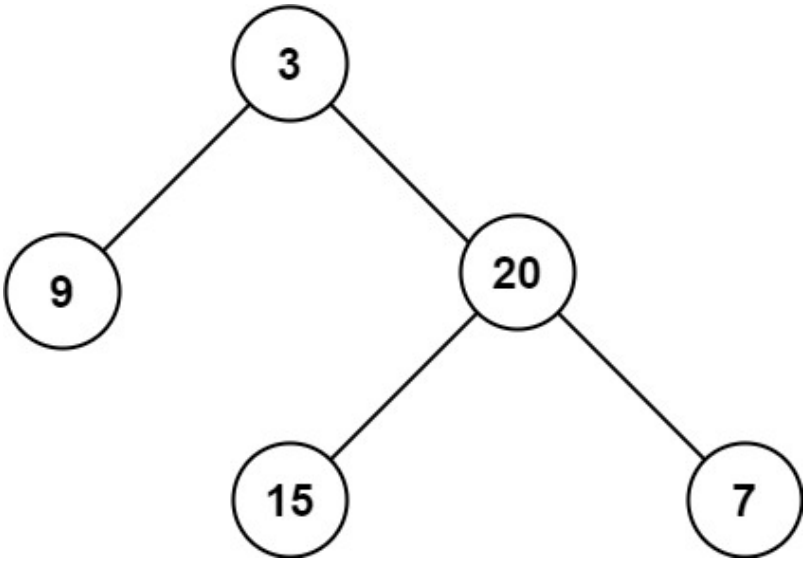
104 Maximum Depth of Binary Tree (link)

Description

Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 3
```

Example 2:

```
Input: root = [1,null,2]
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-100 <= Node.val <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int maxDepth(TreeNode root) {
        if(root==null) return 0;
        return 1+Math.max(maxDepth(root.left),maxDepth(root.right));

    }
}
```

209 Minimum Size Subarray Sum (link)

Description

Given an array of positive integers `nums` and a positive integer `target`, return *the **minimal length** of a subarray whose sum is greater than or equal to* `target`. If there is no such subarray, return `0` instead.

Example 1:

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.
```

Example 2:

```
Input: target = 4, nums = [1,4,4]
Output: 1
```

Example 3:

```
Input: target = 11, nums = [1,1,1,1,1,1,1,1]
Output: 0
```

Constraints:

- `1 <= target <= 10^9`
- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^4`

Follow up: If you have figured out the `O(n)` solution, try coding another solution of which the time complexity is `O(n log(n))`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

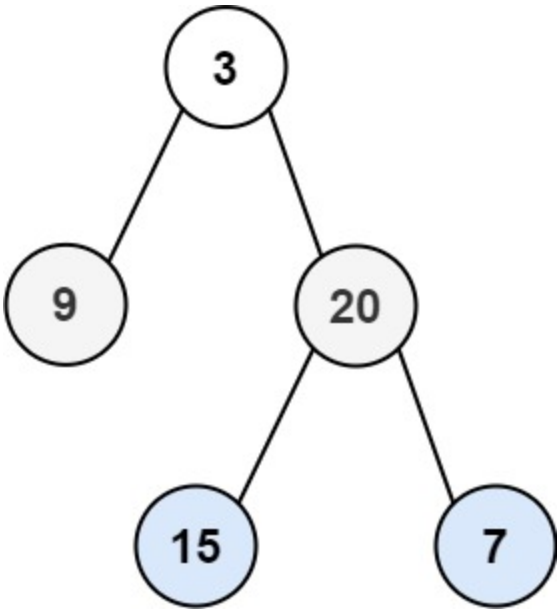
```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int i=0,j=0;
        int ans=Integer.MAX_VALUE,sum=0;
        while(i<=j && j<nums.length){
        while(sum<target && j<nums.length){
            sum+=nums[j++];

        }
        while(sum>=target && i<=j) {
        ans=Math.min(ans,j-i);
           sum-=nums[i];
            i++;
        }

        }
        return ans == Integer.MAX_VALUE ? 0 : ans;



    }
}
```

102 Binary Tree Level Order Traversal (link)

Description

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-1000 <= Node.val <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

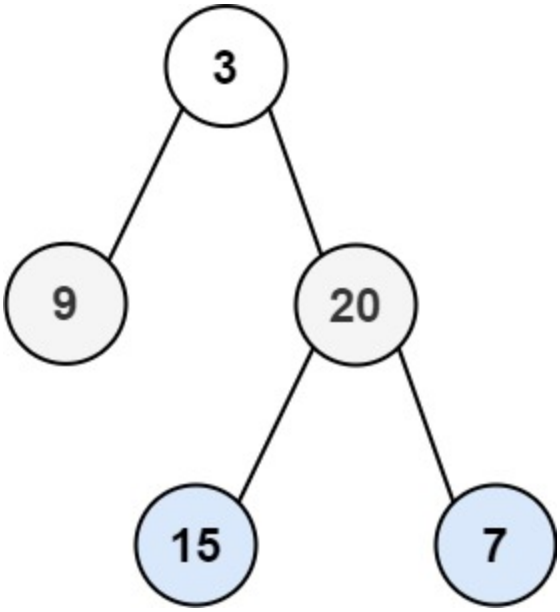
```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) { int f=0;
        List<List<Integer>> ans =new ArrayList<>();
        Queue<TreeNode> q= new LinkedList<>();
        if(root==null)
            return ans;
        q.add(root);
        while(!q.isEmpty()){
            int size=q.size();
            List<Integer> tl= new ArrayList<Integer>();
            for(int i=0;i<size;i++){
                TreeNode temp=q.poll();
                if(temp.left!=null)
                    q.add(temp.left);
                if(temp.right!=null)
                    q.add(temp.right);
                tl.add(temp.val);
            }

                ans.add(tl);

        }

        return ans;


    }
}
```

103 Binary Tree Zigzag Level Order Traversal (link)

Description

Given the `root` of a binary tree, return *the zigzag level order traversal of its nodes' values*. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-100 <= Node.val <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        int f=0;
        List<List<Integer>> ans =new ArrayList<>();
        Queue<TreeNode> q= new LinkedList<>();
        if(root==null)
            return ans;
        q.add(root);
        while(!q.isEmpty()){
            int size=q.size();
            List<Integer> tl= new ArrayList<Integer>();
            for(int i=0;i<size;i++){
                TreeNode temp=q.poll();
                if(temp.left!=null)
                    q.add(temp.left);
                if(temp.right!=null)
                    q.add(temp.right);
                tl.add(temp.val);
            }
            if(f==1){

                Collections.reverse(tl);
                f=0;

            }else{
                f=1;
            }
                ans.add(tl);

        }

        return ans;

    }
}
```

38 Count and Say (link)

Description

The **count-and-say** sequence is a sequence of digit strings defined by the recursive formula:

- `countAndSay(1) = "1"`
- `countAndSay(n)` is the run-length encoding of `countAndSay(n - 1)`.

Run-length encoding (RLE) is a string compression method that works by replacing consecutive identical characters (repeated 2 or more times) with the concatenation of the character and the number marking the count of the characters (length of the run). For example, to compress the string `"3322251"` we replace `"33"` with `"23"`, replace `"222"` with `"32"`, replace `"5"` with `"15"` and replace `"1"` with `"11"`. Thus the compressed string becomes `"23321511"`.

Given a positive integer `n`, return *the* n^{th} *element of the* ***count-and-say*** *sequence*.

Example 1:

Input: n = 4

Output: "1211"

Explanation:

```
countAndSay(1) = "1"
countAndSay(2) = RLE of "1" = "11"
countAndSay(3) = RLE of "11" = "21"
countAndSay(4) = RLE of "21" = "1211"
```

Example 2:

Input: n = 1

Output: "1"

Explanation:

This is the base case.

Constraints:

- `1 <= n <= 30`

Follow up: Could you solve it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {



    public String countAndSay(int n) {

  if (n == 1) return "1";

        String prev = countAndSay(n - 1);
        StringBuilder sb = new StringBuilder();

        int i = 0;
        while (i < prev.length()) {
            char currentChar = prev.charAt(i);
            int count = 1;

            // Count repetitions of currentChar
            while (i + 1 < prev.length() && prev.charAt(i + 1) == currentChar) {
                count++;
                i++;
            }

            sb.append(count).append(currentChar);
            i++;
        }

        return sb.toString();




    }
}
```

11 Container With Most Water (link)

Description

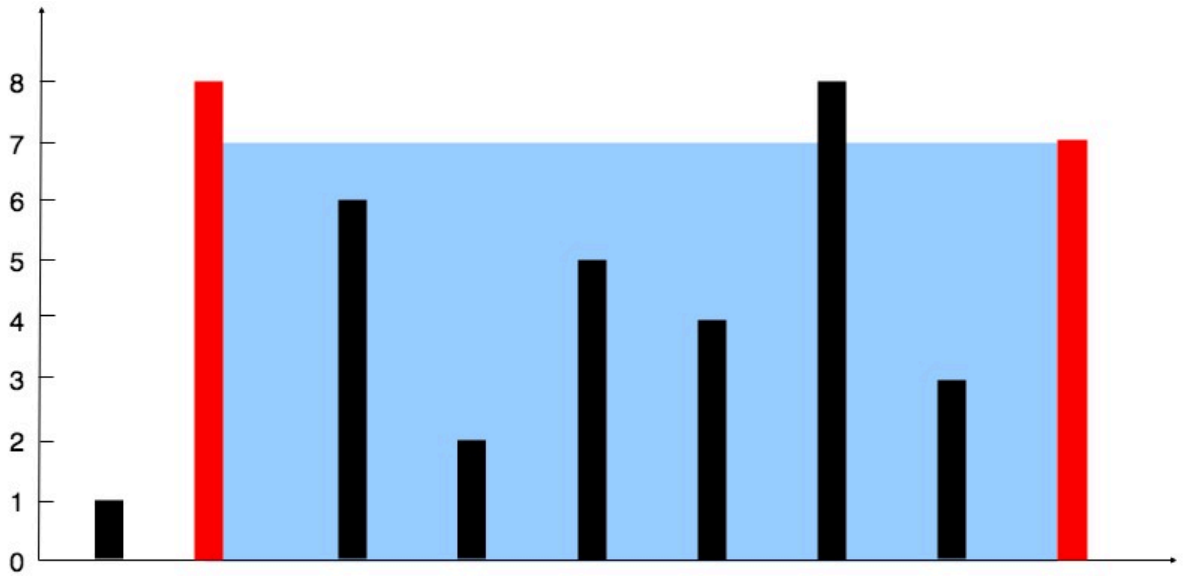
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:




```
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In th
```

Example 2:

```
Input: height = [1,1]
Output: 1
```

Constraints:

- `n == height.length`
- `2 <= n <= 10^5`

- `0 <= height[i] <= 10^4`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxArea(int[] height) {
        int maxarea=0;
        int i=0,j=height.length-1;
        while(i<j){

            maxarea=Math.max(maxarea,Math.min(height[i],height[j])*(j-i));
            if(height[i]>height[j])
            j--;
            else
            i++;
        }
        return maxarea;

    }
}
```

1 Two Sum (link)

Description

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

Constraints:

- $2 <= nums.length <= 10^4$
- $-10^9 <= nums[i] <= 10^9$
- $-10^9 <= target <= 10^9$
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        saveValueIndex = {}
        for index1 in range(0, len(nums)):
            saveValueIndex[nums[index1]] = index1
        pair = []
        for index1 in range(0, len(nums)):
            if target-nums[index1] in saveValueIndex and saveValueIndex[target-nums[index
                pair+=[index1, saveValueIndex[target-nums[index1]]]
                break

        return pair
```

15 3Sum (link)

Description

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.
```

Example 2:

```
Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.
```

Example 3:

```
Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.
```

Constraints:

- `3 <= nums.length <= 3000`
- `-10^5 <= nums[i] <= 10^5`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> ans = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            // Skip duplicate values of i
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            int j = i + 1, k = nums.length - 1;

            while (j < k) {
                int sum = nums[i] + nums[j] + nums[k];

                if (sum == 0) {
                    ans.add(Arrays.asList(nums[i], nums[j], nums[k]));

                    // Skip duplicates for j and k
                    while (j < k && nums[j] == nums[j + 1]) j++;
                    while (j < k && nums[k] == nums[k - 1]) k--;

                    j++;
                    k--;
                } else if (sum < 0) {
                    j++;
                } else {
                    k--;
                }
            }
        }

        return ans;
    }
}
```

198 House Robber (link)

Description

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police***.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

Example 2:

```
Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1)
Total amount you can rob = 2 + 9 + 1 = 12.
```

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 400`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int ans=0;
    int util(int [] n, int st, int end,int[] dp){
        if(end==0)
        return n[0];
        if(end==1)
        return Math.max(n[0],n[1]);
        if(dp[end]!=-1)
        return dp[end];
        return  dp[end]=Math.max(util(n,0,end-1,dp),n[end]+util(n,0,end-2,dp));

    }
    public int rob(int[] nums) {
        int [] dp= new int[nums.length];
        Arrays.fill(dp,-1);

        return util(nums,0,nums.length-1,dp);

    }
}
```

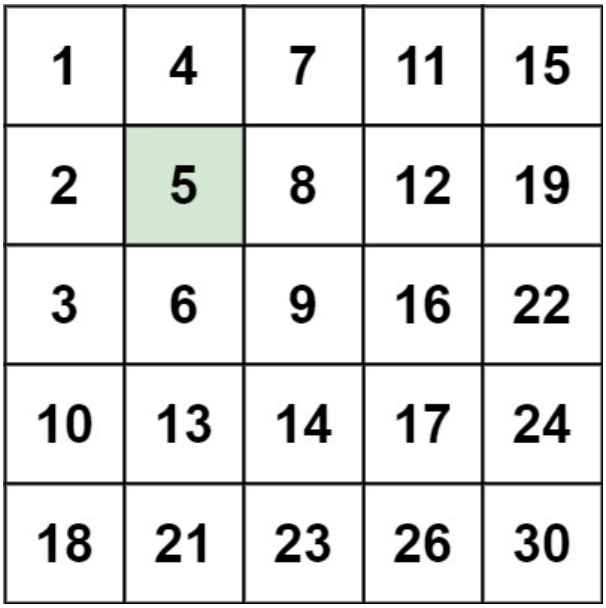
240 Search a 2D Matrix II (link)

Description

Write an efficient algorithm that searches for a value `target` in an `m x n` integer matrix `matrix`. This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:




```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,
Output: true
```

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,
Output: false
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= n, m <= 300`
- -10^9 `<= matrix[i][j] <=` 10^9
- All the integers in each row are **sorted** in ascending order.
- All the integers in each column are **sorted** in ascending order.
- -10^9 `<= target <=` 10^9

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
        public boolean util(int[][] m, int r1,int r2,int c1,int c2,int target) {
            if (r1 > r2 || c1 > c2)
            return false;

        int midr = r1 + (r2 - r1) / 2;
        int midc = c1 + (c2 - c1) / 2;

        if (target == m[midr][midc])
            return true;

        if (m[midr][midc] < target) {
            return util(m, midr + 1, r2, c1, c2, target) ||  // bottom
                   util(m, r1, r2, midc + 1, c2, target);    // right
        } else {
            return util(m, r1, midr - 1, c1, c2, target) ||  // top
                   util(m, r1, r2, c1, midc - 1, target);    // left
        }

        }


    public boolean searchMatrix(int[][] matrix, int target) {
        int r2=matrix.length-1;
        int c2=matrix[0].length-1;

       return  util(matrix,0,r2,0,c2,target);


    }
}
```

3 Longest Substring Without Repeating Characters (link)

Description

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 10^4`
- `s` consists of English letters, digits, symbols and spaces.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int ans=0;
        int i=0,j=0;
        HashMap<Character,Integer> m= new HashMap<>();
        while(i<=j && j<s.length()){
            if(m.containsKey(s.charAt(j)))
             i= Math.max(i, m.get(s.charAt(j))+1);


                           m.put(s.charAt(j),j);

               ans=Math.max(ans,j-i+1);
                j++;

        }
        return ans;


    }
}
```

74 Search a 2D Matrix (link)

Description

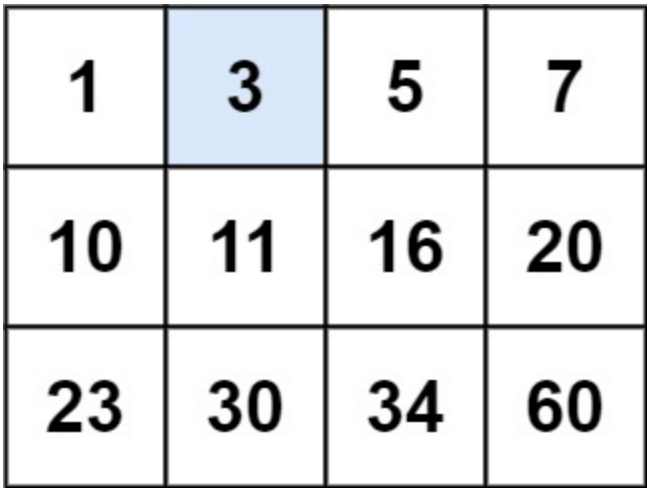
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if* `target` *is in* `matrix` *or* `false` *otherwise*.

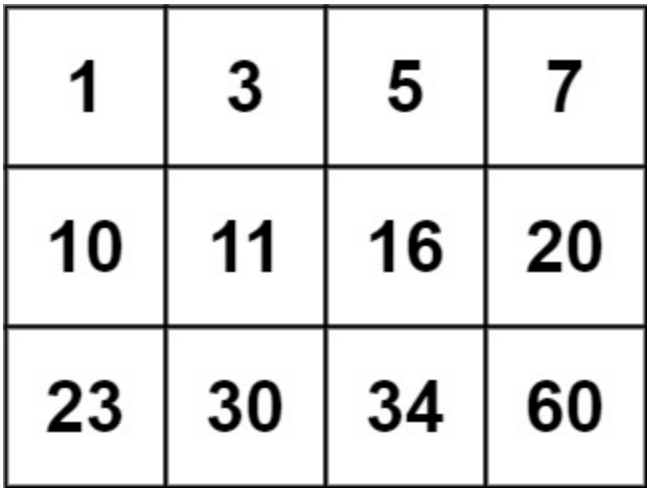
You must write a solution in `O(log(m * n))` time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Example 2:



```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false
```

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 100
- -10^4 <= matrix[i][j], target <= 10^4

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
   int  sm(int [][] matrix,int target,int st,int end){
       if(st>end)
            return Math.max(0, end); // Prevent negative row index


       int mid=st+(end-st)/2;
     if( matrix[mid][0]>target)
    return     sm(matrix,target,st,mid-1);
         else if( matrix[mid][0]<target)
             return         sm(matrix,target,mid+1,end);
             else
                 return mid;




   }
      int  smc(int [][] matrix,int target,int row,int st,int end){
       if(end<st)
           return -1;
       int mid=st+(end-st)/2;
     if( matrix[row][mid]>target)
        return smc(matrix,target,row,st,mid-1);
         else if( matrix[row][mid]<target)
                 return     smc(matrix,target,row,mid+1,end);
             else
                 return mid;



   }
    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0)
            return false;
        int m=matrix.length-1;
        int n=matrix[0].length-1;
         int row=sm(matrix,target,0,m);



         int col=smc(matrix,target,row,0,n);
           if( col!=-1 && matrix[row][col]==target)
               return true;
           else return false;




    }
}
```

1127 Last Stone Weight (link)

Description

You are given an array of integers `stones` where `stones[i]` is the weight of the i^{th} stone.

We are playing a game with the stones. On each turn, we choose the **heaviest two stones** and smash them together. Suppose the heaviest two stones have weights `x` and `y` with `x <= y`. The result of this smash is:

- If `x == y`, both stones are destroyed, and
- If `x != y`, the stone of weight `x` is destroyed, and the stone of weight `y` has new weight `y - x`.

At the end of the game, there is **at most one** stone left.

Return *the weight of the last remaining stone*. If there are no stones left, return `0`.

Example 1:

```
Input: stones = [2,7,4,1,8,1]
Output: 1
Explanation:
We combine 7 and 8 to get 1 so the array converts to [2,4,1,1,1] then,
we combine 2 and 4 to get 2 so the array converts to [2,1,1,1] then,
we combine 2 and 1 to get 1 so the array converts to [1,1,1] then,
we combine 1 and 1 to get 0 so the array converts to [1] then that's the value of the las
```

Example 2:

```
Input: stones = [1]
Output: 1
```

Constraints:

- `1 <= stones.length <= 30`
- `1 <= stones[i] <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> q= new PriorityQueue<>((a,b)->b-a);
        for(int n:stones){
            q.add(n);
        }
        while(q.size()>1){
            int t=q.poll();
            int q1=q.poll();
            if(t-q1>0)
            q.add(t-q1);

        }
        if(q.size()==1)
        return q.peek();
        else
        return 0;

    }
}
```

202 Happy Number (link)

Description

Write an algorithm to determine if a number `n` is happy.

A **happy number** is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it **loops endlessly in a cycle** which does not include 1.
- Those numbers for which this process **ends in 1** are happy.

Return `true` *if* `n` *is a happy number, and* `false` *if not*.

Example 1:

```
Input: n = 19
Output: true
Explanation:
1^2 + 9^2 = 82
8^2 + 2^2 = 68
6^2 + 8^2 = 100
1^2 + 0^2 + 0^2 = 1
```

Example 2:

```
Input: n = 2
Output: false
```

Constraints:

- `1 <= n <= 2^31 - 1`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {


    HashSet<Integer> h= new HashSet<>();


    public boolean isHappy(int n) {
        int r=0;
        while(n>0){
            r+=(n%10)*(n%10);
            n=n/10;
        }
        if(h.contains(r))
        return false;
        h.add(r);
        if(r==1)
        return true;

      return  isHappy(r);



    }
}
```

167 Two Sum II - Input Array Is Sorted (link)

Description

Given a **1-indexed** array of integers `numbers` that is already ***sorted in non-decreasing order***, find two numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where `1 <= index1 < index2 <= numbers.length`.

Return *the indices of the two numbers,* `index1` *and* `index2`*,* ***added by one*** *as an integer array* `[index1, index2]` *of length 2.*

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

Example 1:

```
Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2]
```

Example 2:

```
Input: numbers = [2,3,4], target = 6
Output: [1,3]
Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].
```

Example 3:

```
Input: numbers = [-1,0], target = -1
Output: [1,2]
Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2
```

Constraints:

- `2 <= numbers.length <= 3 * 10^4`
- `-1000 <= numbers[i] <= 1000`
- `numbers` is sorted in **non-decreasing order**.
- `-1000 <= target <= 1000`
- The tests are generated such that there is **exactly one solution**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] twoSum(int[] a, int target) {
        int [] ans=new int[2];
        int i=0,j=a.length-1;
        while(i<j){
            if(a[i]+a[j]<target)
             i++;
            else if(a[i]+a[j]>target)
                    j--;
                    else
                    {
                        ans[0]=i+1;
                        ans[1]=j+1;
                        return ans;
                    }
        }
        return ans;

    }
}
```

215 Kth Largest Element in an Array (link)

Description

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Constraints:

- $1 <= k <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> q=new PriorityQueue<>();
        for(int i=0;i<nums.length;i++){
            q.add(nums[i]);
            if(q.size()>k)
                q.poll();
        }
        return q.peek();

    }
}
```

124 Binary Tree Maximum Path Sum (link)

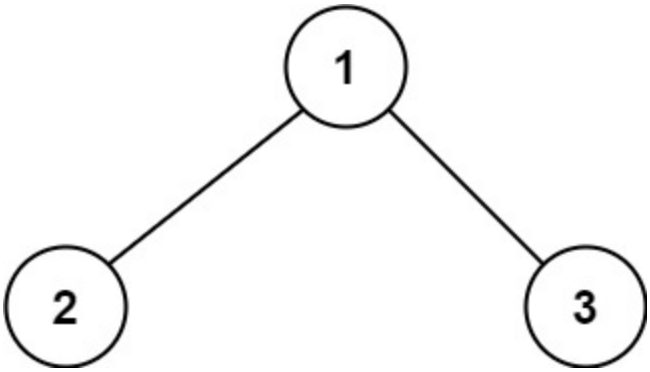
Description

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

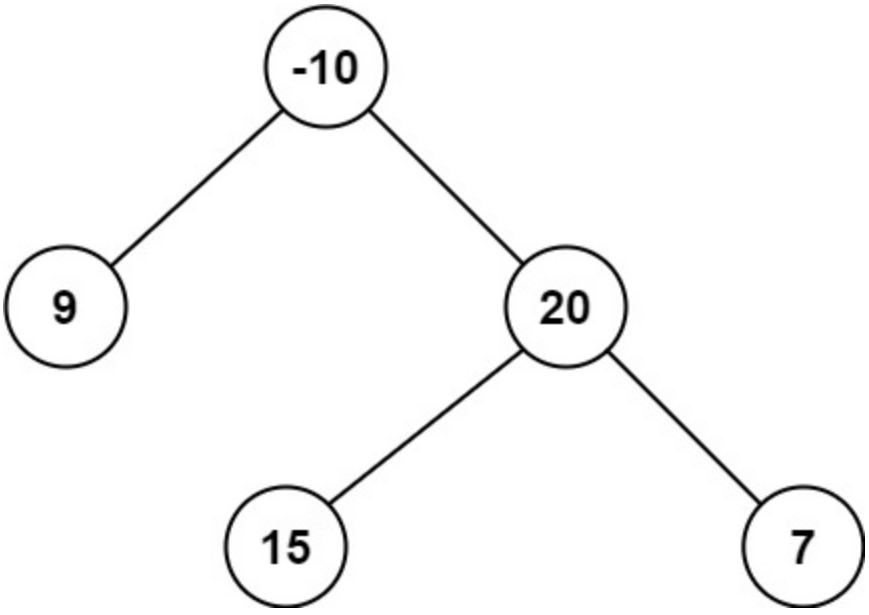
Given the `root` of a binary tree, return *the maximum **path sum** of any **non-empty** path*.

Example 1:




```
Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.
```

Example 2:



```
Input: root = [-10,9,20,null,null,15,7]
Output: 42
Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.
```

Constraints:

- The number of nodes in the tree is in the range `[1, 3 * 10^4]`.
- `-1000 <= Node.val <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int maxsm = Integer.MIN_VALUE;
        public int height(TreeNode root) {
            if(root==null)
            return 0;
          int  ls=Math.max(0,height(root.left));
          int rs=Math.max(0,height(root.right));
          maxsm=Math.max(maxsm,ls+rs+root.val);
        return root.val+Math.max(ls,rs);
        }


    public int maxPathSum(TreeNode root) {

         height(root);
        return maxsm;


    }
}
```

98 Validate Binary Search Tree (link)

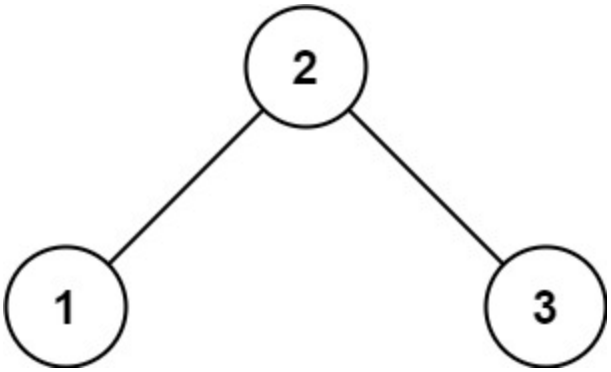
Description

Given the `root` of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

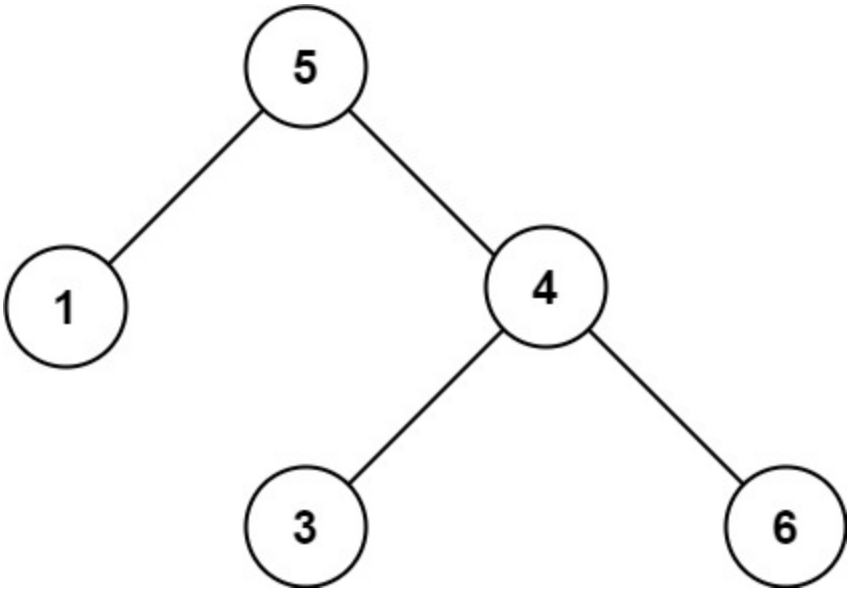
- The left subtree of a node contains only nodes with keys **strictly less than** the node's key.
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [2,1,3]
Output: true
```

Example 2:



```
Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-2^{31} <= Node.val <= 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
     private boolean isValidBST(TreeNode node, long min, long max) {
        if (node == null) return true;

        if (node.val <= min || node.val >= max) return false;

        return isValidBST(node.left, min, node.val) &&
               isValidBST(node.right, node.val, max);
    }

    public boolean isValidBST(TreeNode root) {
             return isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE);


    }
}
```

543 Diameter of Binary Tree (link)

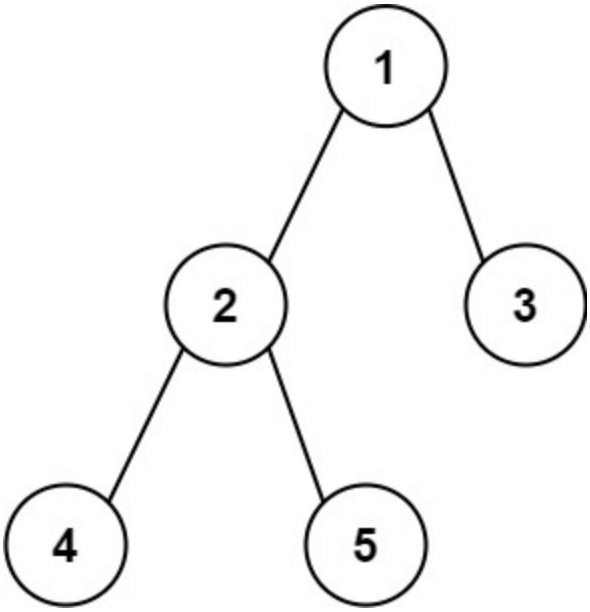
Description

Given the `root` of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the `root`.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:




```
Input: root = [1,2,3,4,5]
Output: 3
Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].
```

Example 2:

```
Input: root = [1,2]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- `-100 <= Node.val <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int maxDiameter=0;
     public int height(TreeNode root) {
        if (root == null) {
            return 0;  // Height of empty tree is 0
        }

        int leftHeight = height(root.left);
        int rightHeight = height(root.right);
                  maxDiameter = Math.max(maxDiameter, leftHeight + rightHeight);

        return 1 + Math.max(leftHeight, rightHeight);
    }


    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
       return maxDiameter ;

    }
}
```

322 Coin Change (link)

Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

Example 3:

```
Input: coins = [1], amount = 0
Output: 0
```

Constraints:

- `1 <= coins.length <= 12`
- `1 <= coins[i] <= 2^31 - 1`
- `0 <= amount <= 10^4`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int coinChange(int[] coins, int amount) {

          int dp[][] =new int[coins.length+1][amount+1];
        for(int [] arr:dp){
            Arrays.fill(arr,-1);
        }
        // Initialize
        for (int i = 0; i <= coins.length; i++) {
            for (int j = 0; j <= amount; j++) {
                if (j == 0) dp[i][j] = 0;
                else if (i == 0) dp[i][j] = (int) 1e9; // simulate infinity
            }
        }
         for(int i=1;i<=coins.length;i++){
            for(int j=1;j<=amount;j++){
                if(coins[i-1]<=j){
                 dp[i][j]=Math.min(dp[i-1][j],1+dp[i][j-coins[i-1]]);
                }else{
                    dp[i][j]=dp[i-1][j];
                }


            }
         }
                return dp[coins.length][amount] >= (int) 1e9 ? -1 : dp[coins.length][amou

    }
}
```

297 Serialize and Deserialize Binary Tree (link)

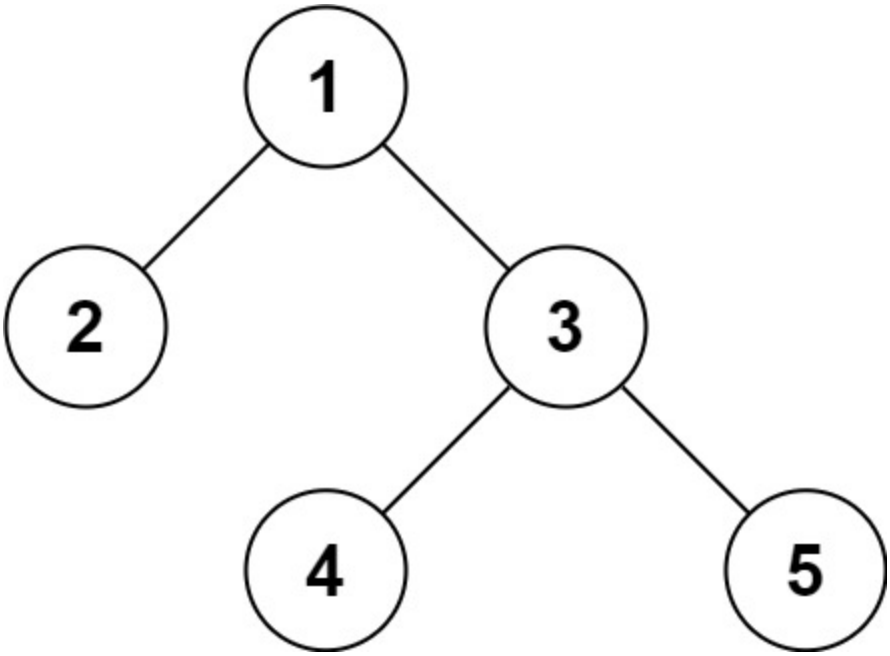
Description

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:




```
Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 10^4]`.
- `-1000 <= Node.val <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {

    // Encodes a tree to a single string.
     public StringBuilder serutil(TreeNode root,StringBuilder sb) {
         if(root==null){
             sb.append("N#");
             return sb;
         }
            sb.append(root.val+"#");
            serutil(root.left,sb);
            serutil(root.right,sb);
         return sb;

     }

    public String serialize(TreeNode root) {
        StringBuilder sb=new StringBuilder("");
        sb=serutil(root,sb);
        return sb.toString();

    }

    // Decodes your encoded data to tree.
       private int i = 0;
        public TreeNode deserializeutil(String [] arr) {
        if (i >= arr.length || arr[i].equals("N")) {
                  i++;
                   return null;
              }
             TreeNode    temp= new TreeNode(Integer.parseInt(arr[i++]));
                 temp.left=deserializeutil(arr);
                 temp.right=deserializeutil(arr);
        return temp;

        }

    public TreeNode deserialize(String data) {
        String [] arr=data.split("#");
        return deserializeutil(arr);

    }
}

// Your Codec object will be instantiated and called as such:
// Codec ser = new Codec();
```

```
// Codec deser = new Codec();
// TreeNode ans = deser.deserialize(ser.serialize(root));
```

20 Valid Parentheses (link)

Description

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "(]"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- `1 <= s.length <= 10^4`
- `s` consists of parentheses only `'()[]{}'`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isValid(String s) {

        Stack<Character> st=new Stack<>();
        for(int i=0;i<s.length();i++){
            if(s.charAt(i)=='(' || s.charAt(i)=='{' || s.charAt(i)=='[')
                st.push(s.charAt(i));
            else if(s.charAt(i)==')' || s.charAt(i)=='}' || s.charAt(i)==']'){
                if(st.isEmpty())
                    return false;
                if(s.charAt(i)==')' && st.peek()!='(')
                    return false;

                   if(s.charAt(i)=='}' && st.peek()!='{')
                    return false;
                 if(s.charAt(i)==']' && st.peek()!='[')
                    return false;
             st.pop();

            }


        }
        if(st.isEmpty())
            return true;
        else
            return false;

    }
}
```

25 Reverse Nodes in k-Group (link)

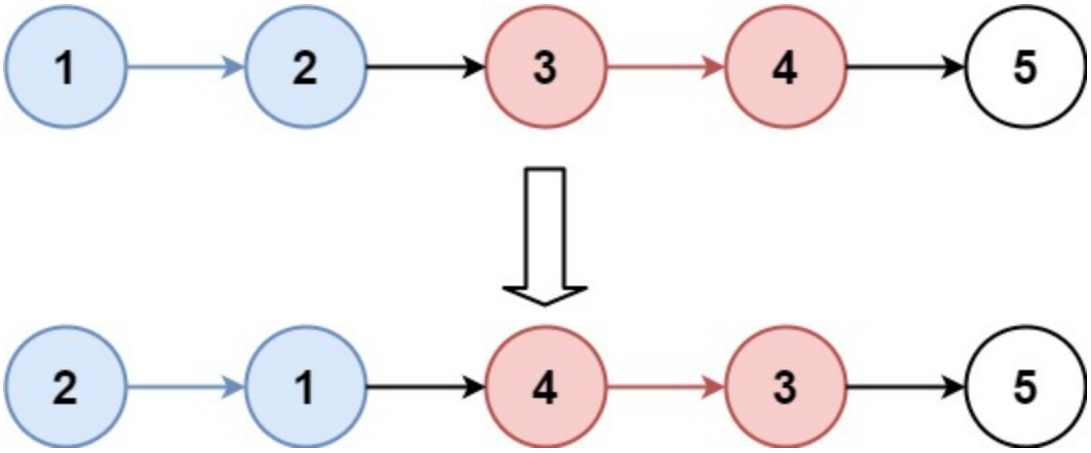
Description

Given the `head` of a linked list, reverse the nodes of the list `k` at a time, and return *the modified list*.

`k` is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of `k` then left-out nodes, in the end, should remain as it is.

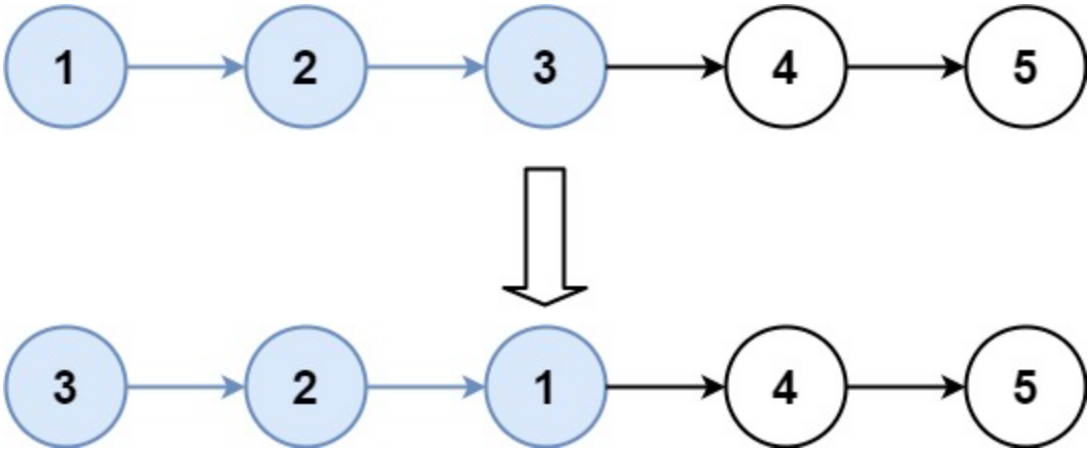
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
```

Example 2:




```
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]
```

Constraints:

- The number of nodes in the list is `n`.
- `1 <= k <= n <= 5000`
- `0 <= Node.val <= 1000`

Follow-up: Can you solve the problem in `O(1)` extra memory space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode knode(ListNode temp, int k){
        k--;
        while(temp!=null && k>0){
            temp=temp.next;
            k--;
        }
        return temp;
    }
   public ListNode reversell(ListNode temp){
         ListNode curr=temp;
       ListNode prev=null;
        while(curr!=null){
            ListNode nex=curr.next;
            curr.next=prev;
            prev=curr;
            curr=nex;
        }
       return prev;

     }

    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode temp=head;
        ListNode prevgroup=null;
        while(temp!=null){

        ListNode  kthnode=knode(temp, k);
            if(kthnode==null && null!=prevgroup){
                prevgroup.next=temp;
                break;
            }
            ListNode kthnextnode=kthnode.next;
            kthnode.next=null;
         ListNode ka=reversell(temp);
            if(temp==head){
                head=kthnode;
            }else{
            prevgroup.next=kthnode;
        }

            // Update the pointer to the
            // last node of the previous group
            prevgroup = temp;
            temp=kthnextnode;
    }
```

```
        return head;
    }
}
```

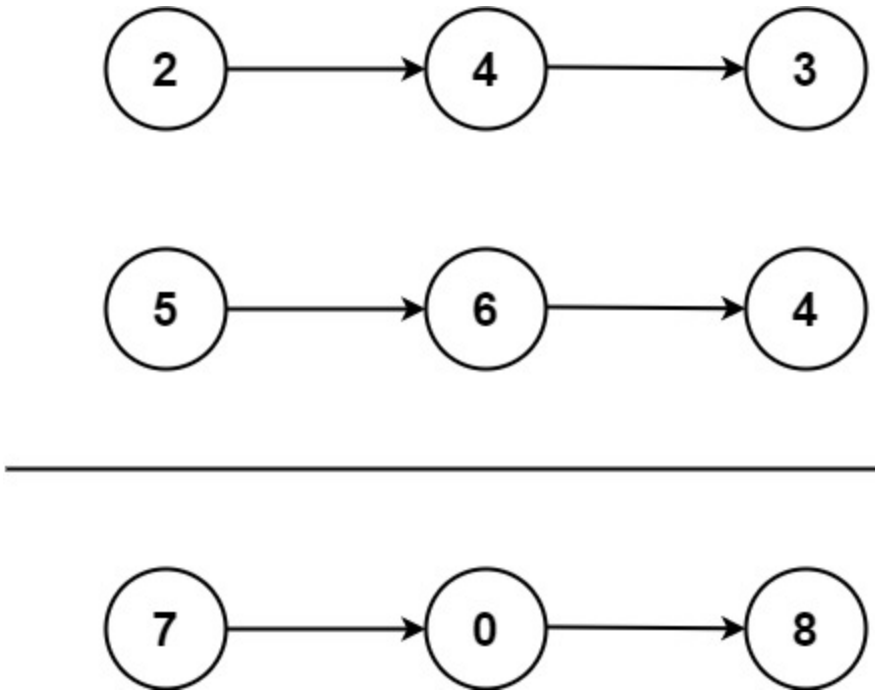
2 Add Two Numbers (link)

Description

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.
```

Example 2:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

Example 3:

```
Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Output: [8,9,9,9,0,0,0,1]
```

Constraints:

- The number of nodes in each linked list is in the range `[1, 100]`.
- `0 <= Node.val <= 9`
- It is guaranteed that the list represents a number that does not have leading zeros.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode anshead=null;
        ListNode temp=null;
            int c=0;
            int val=0;
        while(l1 !=null && l2!=null){
            int sm=l1.val+l2.val+c;
            val=sm%10;
            c=sm/10;
            if(temp==null){
               temp=new ListNode(val);
                anshead=temp;
            }else{
                temp.next=new ListNode(val);
                temp = temp.next;
            }
            l1=l1.next;
            l2=l2.next;

        }
        while(l1!=null){
              int sm=l1.val+c;
            val=sm%10;
            c=sm/10;
              temp.next=new ListNode(val);
              l1=l1.next;
            temp = temp.next;

        }
        while(l2!=null){
             int sm=l2.val+c;
            val=sm%10;
            c=sm/10;
              temp.next=new ListNode(val);

              l2=l2.next;
            temp = temp.next;
        }
        if(c != 0) {
            temp.next = new ListNode(c);
        }
        return anshead;
```

```
        }
    }
```

854 Making A Large Island (link)

Description

You are given an `n x n` binary matrix `grid`. You are allowed to change **at most one** `0` to be `1`.

Return *the size of the largest **island** in* `grid` *after applying this operation*.

An **island** is a 4-directionally connected group of `1`s.

Example 1:

```
Input: grid = [[1,0],[0,1]]
Output: 3
Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.
```

Example 2:

```
Input: grid = [[1,1],[1,0]]
Output: 4
Explanation: Change the 0 to 1 and make the island bigger, only one island with area = 4
```

Example 3:

```
Input: grid = [[1,1],[1,1]]
Output: 4
Explanation: Can't change any 0 to 1, only one island with area = 4.
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 500`
- `grid[i][j]` is either `0` or `1`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
from collections import deque

class Solution:
    def largestIsland(self, grid: List[List[int]]) -> int:
        n = len(grid)
        traversed = []
        onesQueue = deque([])
        for row in range(0, n):
            currList = []
            for col in range(0, n):
                currList += [None]
                if grid[row][col]==1:
                    onesQueue.append([row,col])
            traversed += [currList]

        cellsToTraverse = [[-1,0],[0,1],[1,0],[0,-1]]
        currClusterIndex = 1
        while len(onesQueue)>0:
            currOne = onesQueue.popleft()
            if traversed[currOne[0]][currOne[1]]:
                continue
            traversed[currOne[0]][currOne[1]] = currClusterIndex
            currQueue = deque([currOne])
            while len(currQueue)>0:
                loc = currQueue.popleft()
                for cell in cellsToTraverse:
                    newX = loc[0]+cell[0]
                    newY = loc[1]+cell[1]
                    if newX>=0 and newX<n and newY>=0 and newY<n and not traversed[newX]
                        traversed[newX][newY] = currClusterIndex
                        currQueue.append([newX, newY])
            currClusterIndex += 1
        clusterCounts = {}
        for row in range(0, n):
            for col in range(0, n):
                if traversed[row][col]:
                    if traversed[row][col] not in clusterCounts:
                        clusterCounts[traversed[row][col]] = 1
                    else:
                        clusterCounts[traversed[row][col]] += 1
        maxNumber = 0
        for cluster in clusterCounts:
            maxNumber = max(maxNumber, clusterCounts[cluster])
        for row in range(0, n):
            for col in range(0, n):
                if not traversed[row][col]:
                    clustersFound = []
                    for cell in cellsToTraverse:
                        newX = row+cell[0]
                        newY = col+cell[1]
                        if newX>=0 and newX<n and newY>=0 and newY<n and traversed[newX]
                            clustersFound += [traversed[newX][newY]]
                    clustersFound = set(clustersFound)
                    elemCount = 1
                    for cluster in clustersFound:
                        if cluster in clusterCounts:
```

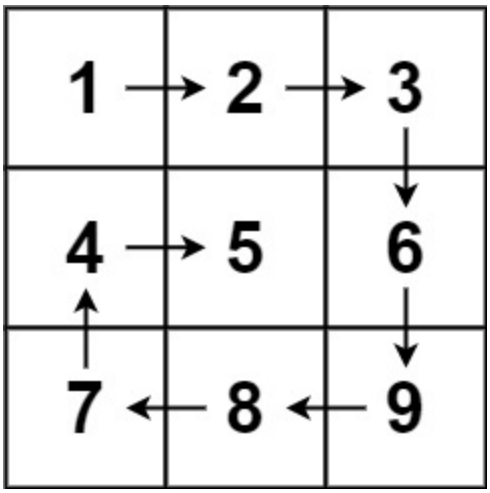
```
                elemCount += clusterCounts[cluster]
            else:
                elemCount = 1
        maxNumber = max(maxNumber, elemCount)
```

54 Spiral Matrix (link)

Description

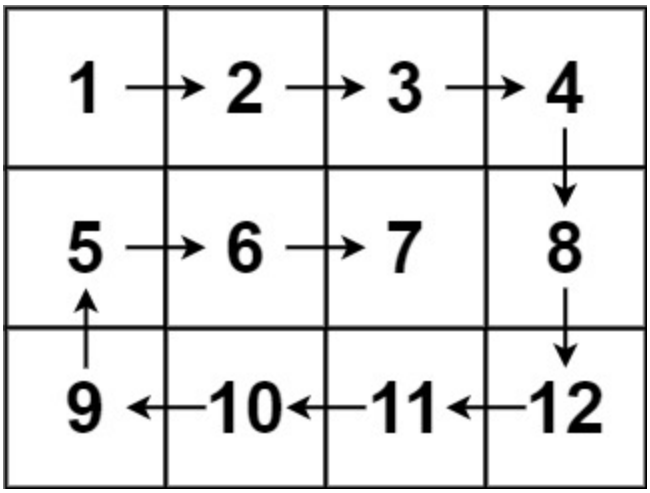
Given an `m x n matrix`, return *all elements of the* `matrix` *in spiral order*.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Example 2:




```
Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]
```

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
from collections import deque
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        output = []
        rows = len(matrix)
        cols = len(matrix[0])
        startRow = 0
        startCol = 0
        endRow = rows-1
        endCol = cols-1
        while startRow<endRow and startCol<endCol:
            #Traverse Top row
            for colIndex in range(startCol, endCol+1):
                output.append(matrix[startRow][colIndex])

            # Traverse Right col
            for rowIndex in range(startRow+1, endRow+1):
                output.append(matrix[rowIndex][endCol])

            # Traverse Bottom row
            for colIndex in range(endCol-1, startCol-1, -1):
                output.append(matrix[endRow][colIndex])

            # Treverse Left col
            for rowIndex in range(endRow-1, startRow, -1):
                output.append(matrix[rowIndex][startCol])

            startCol += 1
            endCol -= 1
            startRow += 1
            endRow -= 1

        if startRow==endRow and startCol==endCol:
            output.append(matrix[startRow][startCol])
        elif startRow==endRow:
            for colIndex in range(startCol, endCol+1):
                output.append(matrix[startRow][colIndex])
        elif startCol==endCol:
            for rowIndex in range(startRow, endRow+1):
                output.append(matrix[rowIndex][startCol])
        return output
```

833 Bus Routes (link)

Description

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the i^{th} bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0^{th} bus travels in the sequence `1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ...` forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return *the least number of buses you must take to travel from* `source` *to* `target`. Return `-1` if it is not possible.

Example 1:

```
Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6
Output: 2
Explanation: The best strategy is take the first bus to the bus stop 7, then take the sec
```

Example 2:

```
Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12
Output: -1
```

Constraints:

- `1 <= routes.length <= 500.`
- `1 <= routes[i].length <=` 10^5
- All the values of `routes[i]` are **unique**.
- `sum(routes[i].length) <=` 10^5
- `0 <= routes[i][j] <` 10^6
- `0 <= source, target <` 10^6

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
from collections import deque
class Solution:
    def numBusesToDestination(self, routes: List[List[int]], source: int, target: int) -:
        if source==target:
            return 0
        stationRouteMap = {}
        routeStationMap = {}
        routeCount = 1
        for route in routes:
            for station in route:
                if station in stationRouteMap:
                    stationRouteMap[station].append(routeCount)
                else:
                    stationRouteMap[station] = [routeCount]
            routeStationMap[routeCount] = set(route)
            routeCount += 1
        for station in stationRouteMap:
            stationRouteMap[station] = set(stationRouteMap[station])

        routesTraversed = {}
        for route in routeStationMap:
            routesTraversed[route] = False
        try:
            routeQueue = deque([[route, 1] for route in stationRouteMap[source]])
            for route in stationRouteMap[source]:
                routesTraversed[route] = True

            while len(routeQueue)>0:
                currRoute = routeQueue.popleft()
                busCount = currRoute[1]
                currRoute = currRoute[0]
                routesTraversed[currRoute] = True
                if target in routeStationMap[currRoute]:
                    return busCount
                for station in routeStationMap[currRoute]:
                    for route in stationRouteMap[station]:
                        if not routesTraversed[route]:
                            routeQueue.append([route, busCount+1])
                            routesTraversed[route] = True
        except Exception as e:
            pass

        return -1
```

2711 Minimum Time to Visit a Cell In a Grid (link)

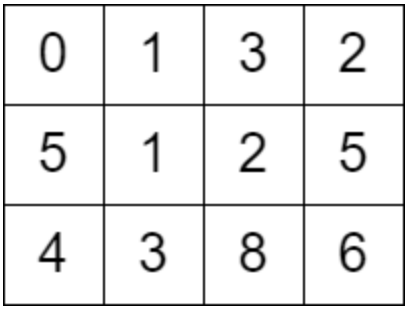
Description

You are given a `m x n` matrix `grid` consisting of **non-negative** integers where `grid[row][col]` represents the **minimum** time required to be able to visit the cell `(row, col)`, which means you can visit the cell `(row, col)` only when the time you visit it is greater than or equal to `grid[row][col]`.

You are standing in the **top-left** cell of the matrix in the 0^{th} second, and you must move to **any** adjacent cell in the four directions: up, down, left, and right. Each move you make takes 1 second.

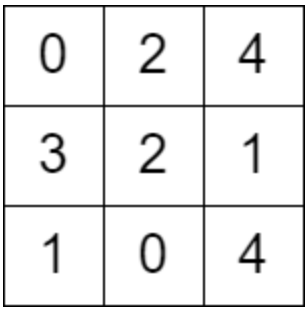
Return *the **minimum** time required in which you can visit the bottom-right cell of the matrix*. If you cannot visit the bottom-right cell, then return `-1`.

Example 1:




```
Input: grid = [[0,1,3,2],[5,1,2,5],[4,3,8,6]]
Output: 7
Explanation: One of the paths that we can take is the following:
- at t = 0, we are on the cell (0,0).
- at t = 1, we move to the cell (0,1). It is possible because grid[0][1] <= 1.
- at t = 2, we move to the cell (1,1). It is possible because grid[1][1] <= 2.
- at t = 3, we move to the cell (1,2). It is possible because grid[1][2] <= 3.
- at t = 4, we move to the cell (1,1). It is possible because grid[1][1] <= 4.
- at t = 5, we move to the cell (1,2). It is possible because grid[1][2] <= 5.
- at t = 6, we move to the cell (1,3). It is possible because grid[1][3] <= 6.
- at t = 7, we move to the cell (2,3). It is possible because grid[2][3] <= 7.
The final time is 7. It can be shown that it is the minimum time possible.
```

Example 2:



```
Input: grid = [[0,2,4],[3,2,1],[1,0,4]]
Output: -1
Explanation: There is no path from the top left to the bottom-right cell.
```

Constraints:

- m == grid.length
- n == grid[i].length
- 2 <= m, n <= 1000
- 4 <= m * n <= 10^5
- 0 <= grid[i][j] <= 10^5
- grid[0][0] == 0

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
import heapq

class Solution:
    def __init__(self):
        self.Queue = []
        self.Calculated = []

    def validCell(self, currRow, currCol, grid):
        rows = len(grid)
        cols = len(grid[0])
        if currRow<rows and currCol<cols and currRow>=0 and currCol>=0:
            return True
        else:
            return False

    def traverse(self, grid):
        rows = len(grid)
        cols = len(grid[0])
        self.Queue = [[0, 0, 0]]
        while len(self.Queue) >0:
            elem = heapq.heappop(self.Queue)
            currRow = elem[1]
            currCol = elem[2]
            time = elem[0]
            self.Calculated[currRow][currCol] = time
            if currRow==len(grid)-1 and currCol==len(grid[0])-1:
                break

            traverseCells = [[-1,0], [1,0], [0, 1],[0, -1]]
            for cell in traverseCells:
                x_new = currRow + cell[0]
                y_new = currCol + cell[1]

                if self.validCell(x_new, y_new, grid) and self.Calculated[x_new][y_new]==
                    if grid[x_new][y_new]<=time:
                        heapq.heappush(self.Queue, [time+1, x_new, y_new])
                        self.Calculated[x_new][y_new] = time+1
                        if x_new==rows-1 and y_new==cols-1:
                            return time+1
                    elif (grid[x_new][y_new]-time)%2:
                        heapq.heappush(self.Queue, [grid[x_new][y_new], x_new, y_new])
                        self.Calculated[x_new][y_new] = grid[x_new][y_new]
                        if x_new==rows-1 and y_new==cols-1:
                            return grid[x_new][y_new]
                    else:
                        heapq.heappush(self.Queue, [grid[x_new][y_new]+1, x_new, y_new])
                        self.Calculated[x_new][y_new] = grid[x_new][y_new]+1
                        if x_new==rows-1 and y_new==cols-1:
                            return grid[x_new][y_new]+1
        return -1

    def minimumTime(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])
        if (rows>1 and grid[0][1]>1) and (cols>1 and grid[1][0]>1):
            return -1
```

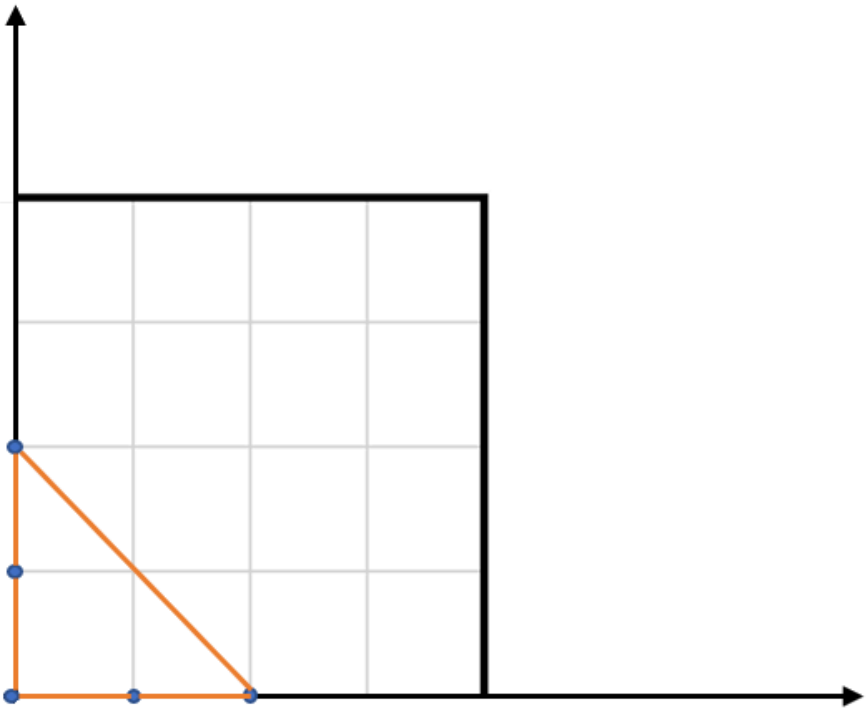
```
        self.Calculated = [[None for col in range(0, cols)] for row in range(0,rows)]
```

830 Largest Triangle Area (link)

Description

Given an array of points on the **X-Y** plane `points` where `points[i] = [x_i, y_i]`, return *the area of the largest triangle that can be formed by any three different points*. Answers within `10^-5` of the actual answer will be accepted.

Example 1:



```
Input: points = [[0,0],[0,1],[1,0],[0,2],[2,0]]
Output: 2.00000
Explanation: The five points are shown in the above figure. The red triangle is the large
```

Example 2:

```
Input: points = [[1,0],[0,0],[0,1]]
Output: 0.50000
```

Constraints:

- `3 <= points.length <= 50`
- `-50 <= x_i, y_i <= 50`

- All the given points are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def areaTriangle(self, point1, point2, point3):
        area = (point1[0]*(point2[1]-point3[1]) + point2[0]*(point3[1]-point1[1]) + point
        #print(point1, point2, point3, area)
        return abs(area)

    def largestTriangleArea(self, points: List[List[int]]) -> float:
        largestArea = 0
        length = len(points)
        for index1 in range(0, length):
            for index2 in range(index1+1, length):
                for index3 in range(index2+1, length):
                    areaTriangle = self.areaTriangle(points[index1], points[index2], poir
                    if areaTriangle>largestArea:
                        largestArea = areaTriangle
        return largestArea
```

1171 Shortest Path in Binary Matrix (link)

Description

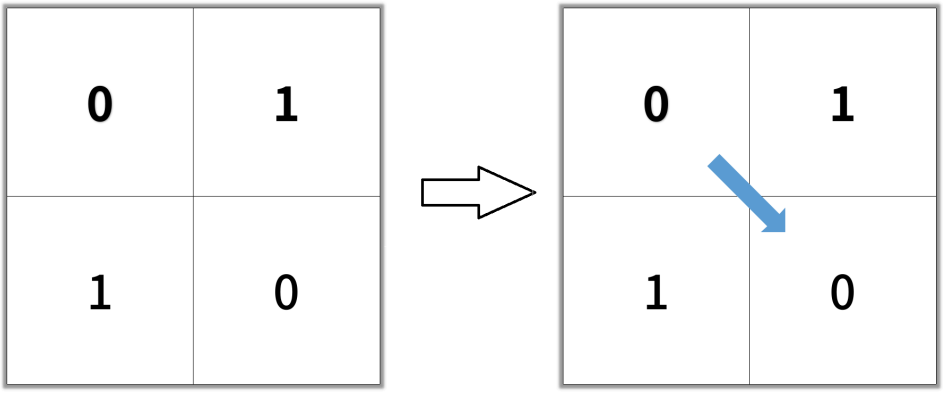
Given an `n x n` binary matrix `grid`, return *the length of the shortest **clear path** in the matrix*. If there is no clear path, return `-1`.

A **clear path** in a binary matrix is a path from the **top-left** cell (i.e., `(0, 0)`) to the **bottom-right** cell (i.e., `(n - 1, n - 1)`) such that:

- All the visited cells of the path are `0`.
- All the adjacent cells of the path are **8-directionally** connected (i.e., they are different and they share an edge or a corner).

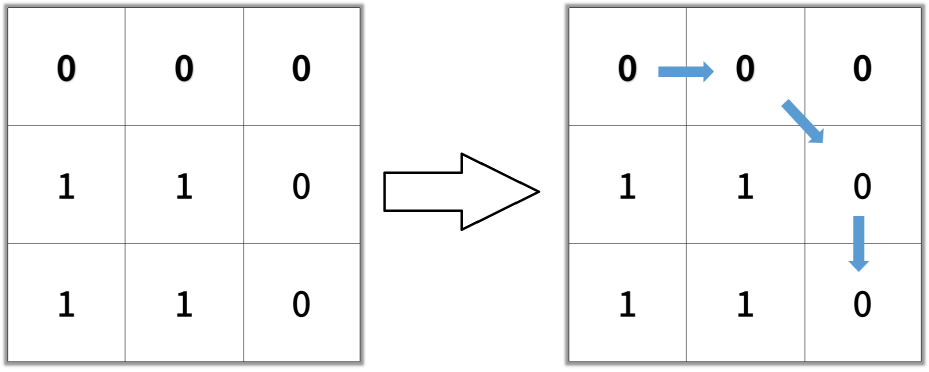
The **length of a clear path** is the number of visited cells of this path.

Example 1:




```
Input: grid = [[0,1],[1,0]]
Output: 2
```

Example 2:



```
Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1
```

Constraints:

- n == grid.length
- n == grid[i].length
- 1 <= n <= 100
- grid[i][j] is 0 or 1

(scroll down for solution)

Solution

Language: python3

Status: Accepted

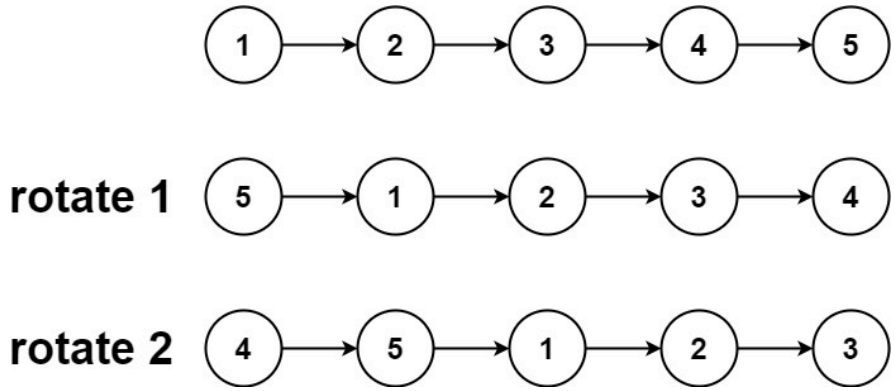
```python
from collections import deque

class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])
        if grid[0][0]==1:
            return -1
        elif rows==1 and cols==1:
            return 1
        traversed = [[False for col in grid[0]] for row in grid]
        traversed[0][0] = True
        queue = deque([[1, 0, 0]])
        paths = [[-1,-1], [-1, 0], [-1, 1], [0, 1], [1, 1], [1, 0], [1, -1], [0, -1]]
        while len(queue)>0:
            elem = queue.popleft()
            pathLength = elem[0]
            currX = elem[1]
            currY = elem[2]

            for path in paths:
                newX = currX + path[0]
                newY = currY + path[1]
                if newX>=0 and newY>=0 and newX<rows and newY<cols and not traversed[newX
                    if newX==rows-1 and newY==cols-1:
                        return pathLength+1

                    queue.append([pathLength+1, newX, newY])
                    traversed[newX][newY] = True

        return -1
```

61 Rotate List (link)

Description

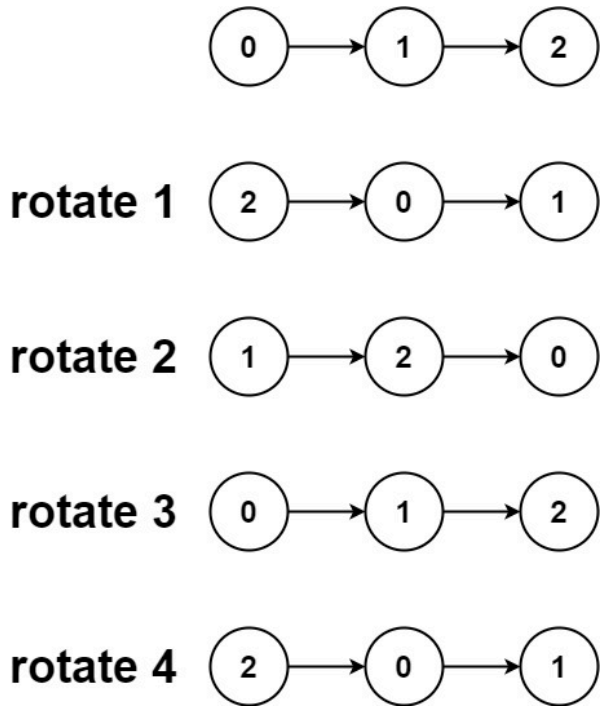
Given the `head` of a linked list, rotate the list to the right by `k` places.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]
```

Example 2:




```
Input: head = [0,1,2], k = 4
Output: [2,0,1]
```

Constraints:

- The number of nodes in the list is in the range `[0, 500]`.
- `-100 <= Node.val <= 100`
- `0 <= k <= 2 * 10^9`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        length = 0
        node = head
        while node:
            node = node.next
            length += 1
        if length==0:
            return head
        k = k%length
        nodesToSkip = length - k
        node = head
        while nodesToSkip-1>0:
            node = node.next
            nodesToSkip -= 1

        if node.next:
            newHead = node.next
            node.next = None
            temp = head
            head = newHead

            while newHead.next:
                newHead = newHead.next

            newHead.next = temp
        return head
```

151 Reverse Words in a String (link)

Description

Given an input string `s`, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in `s` will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space.*

Note that `s` may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"
Output: "blue is sky the"
```

Example 2:

```
Input: s = "  hello world  "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
```

Example 3:

```
Input: s = "a good   example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in th
```

Constraints:

- `1 <= s.length <= 10^4`
- `s` contains English letters (upper-case and lower-case), digits, and spaces `' '`.
- There is **at least one** word in `s`.

Follow-up: If the string data type is mutable in your language, can you solve it **in-place** with `O(1)` extra space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def reverseWords(self, s: str) -> str:
        words = s.split()
        length = len(words)
        i=length-1
        outputString = ""
        while i>-1:
            outputString += words[i]
            i -= 1
            outputString += " "
        return outputString.strip()
```

1894 Merge Strings Alternately (link)

Description

You are given two strings `word1` and `word2`. Merge the strings by adding letters in alternating order, starting with `word1`. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return *the merged string.*

Example 1:

```
Input: word1 = "abc", word2 = "pqr"
Output: "apbqcr"
Explanation: The merged string will be merged as so:
word1:  a   b   c
word2:    p   q   r
merged: a p b q c r
```

Example 2:

```
Input: word1 = "ab", word2 = "pqrs"
Output: "apbqrs"
Explanation: Notice that as word2 is longer, "rs" is appended to the end.
word1:  a   b
word2:    p   q   r   s
merged: a p b q   r   s
```

Example 3:

```
Input: word1 = "abcd", word2 = "pq"
Output: "apbqcd"
Explanation: Notice that as word1 is longer, "cd" is appended to the end.
word1:  a   b   c   d
word2:    p   q
merged: a p b q c   d
```

Constraints:

- `1 <= word1.length, word2.length <= 100`
- `word1` and `word2` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def mergeAlternately(self, word1: str, word2: str) -> str:
        newString = []
        length1 = len(word1)
        length2 = len(word2)
        i=0
        j=0
        while i<length1:
            newString += [word1[i]]
            i += 1
            while j<length2:
                newString += [word2[j]]
                j += 1
                break
        while j<length2:
            newString += [word2[j]]
            j += 1
        return ''.join(newString)
```

581 Shortest Unsorted Continuous Subarray (link)

Description

Given an integer array `nums`, you need to find one **continuous subarray** such that if you only sort this subarray in non-decreasing order, then the whole array will be sorted in non-decreasing order.

Return *the shortest such subarray and output its length*.

Example 1:

```
Input: nums = [2,6,4,8,10,9,15]
Output: 5
Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array
```

Example 2:

```
Input: nums = [1,2,3,4]
Output: 0
```

Example 3:

```
Input: nums = [1]
Output: 0
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^5 <= nums[i] <= 10^5`

Follow up: Can you solve it in `O(n)` time complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        globalStart = len(nums)
        globalEnd = -1
        stack = []
        for index in range(0, len(nums)):
            while len(stack)>0 and nums[index]<nums[stack[-1]]:
                num = stack.pop()
                if globalStart>num:
                    globalStart = num
            stack.append(index)

        stack = []
        for index in range(len(nums)-1, -1, -1):
            while len(stack)>0 and nums[index]>nums[stack[-1]]:
                num = stack.pop()
                if globalEnd<num:
                    globalEnd = num
            stack.append(index)

        #print(stack)
        #print(globalStart, globalEnd)
        if globalEnd<globalStart:
            return 0
        return globalEnd-globalStart+1
```

2227 Sum of Subarray Ranges (link)

Description

You are given an integer array `nums`. The **range** of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return *the **sum of all** subarray ranges of* `nums`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,2,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[2], range = 2 - 2 = 0
[3], range = 3 - 3 = 0
[1,2], range = 2 - 1 = 1
[2,3], range = 3 - 2 = 1
[1,2,3], range = 3 - 1 = 2
So the sum of all ranges is 0 + 0 + 0 + 1 + 1 + 2 = 4.
```

Example 2:

```
Input: nums = [1,3,3]
Output: 4
Explanation: The 6 subarrays of nums are the following:
[1], range = largest - smallest = 1 - 1 = 0
[3], range = 3 - 3 = 0
[3], range = 3 - 3 = 0
[1,3], range = 3 - 1 = 2
[3,3], range = 3 - 3 = 0
[1,3,3], range = 3 - 1 = 2
So the sum of all ranges is 0 + 0 + 0 + 2 + 0 + 2 = 4.
```

Example 3:

```
Input: nums = [4,-2,-3,4,1]
Output: 59
Explanation: The sum of all subarray ranges of nums is 59.
```

Constraints:

- `1 <= nums.length <= 1000`
- `-10^9 <= nums[i] <= 10^9`

Follow-up: Could you find a solution with `O(n)` time complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def subArrayRanges(self, nums: List[int]) -> int:
        # Finding Sum of smallest
        smallestSum = 0
        stack = []
        prevMin = [-1 for i in nums]
        nextMin = [len(nums) for i in nums]
        for index in range(0,len(nums)):
            while len(stack)>0 and nums[stack[-1]]>=nums[index]:
                val = stack.pop()
                nextMin[val] = index
            stack.append(index)
        stack = []
        #print(nextMin)
        for index in range(len(nums)-1, -1, -1):
            while len(stack)>0 and nums[stack[-1]]>nums[index]:
                val = stack.pop()
                prevMin[val] = index
            stack.append(index)
        #print(prevMin)
        for index in range(0,len(nums)):
            smallestSum += (index-prevMin[index])*(nextMin[index]-index)*nums[index]


        # Finding Sum of Largest
        largestSum = 0
        stack = []
        prevMax = [-1 for i in nums]
        nextMax = [len(nums) for i in nums]
        for index in range(0,len(nums)):
            while len(stack)>0 and nums[stack[-1]]<=nums[index]:
                val = stack.pop()
                nextMax[val] = index
            stack.append(index)
        stack = []
        #print(nextMax)
        for index in range(len(nums)-1, -1, -1):
            while len(stack)>0 and nums[stack[-1]]<nums[index]:
                val = stack.pop()
                prevMax[val] = index
            stack.append(index)
        #print(prevMax)
        for index in range(0,len(nums)):
            largestSum += (index-prevMax[index])*(nextMax[index]-index)*nums[index]

        return largestSum-smallestSum
```

560 Subarray Sum Equals K (link)

Description

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to* `k`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,1,1], k = 2
Output: 2
```

Example 2:

```
Input: nums = [1,2,3], k = 3
Output: 2
```

Constraints:

- `1 <= nums.length <= 2 * 10^4`
- `-1000 <= nums[i] <= 1000`
- `-10^7 <= k <= 10^7`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        sums = []
        currSum = 0
        for num in nums:
            currSum += num
            sums.append(currSum)
        #print(sums)
        count = 0
        pervSumMap = {}
        for index in range(0, len(nums)):
            #print(sums[index], sums[index]-k, pervSumMap)
            if (sums[index]-k) in pervSumMap:
                count += len(pervSumMap[sums[index]-k])
            if (sums[index]-k)==0:
                count += 1
            if sums[index] in pervSumMap:
                pervSumMap[sums[index]].append(sums[index])
            else:
                pervSumMap[sums[index]] = [sums[index]]
        return count
```

943 Sum of Subarray Minimums (link)

Description

Given an array of integers arr, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer **modulo** $10^9 + 7$.

Example 1:

```
Input: arr = [3,1,2,4]
Output: 17
Explanation:
Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].
Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.
Sum is 17.
```

Example 2:

```
Input: arr = [11,81,94,43,3]
Output: 444
```

Constraints:

- `1 <= arr.length <= 3 * 10^4`
- `1 <= arr[i] <= 3 * 10^4`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def sumSubarrayMins(self, arr: List[int]) -> int:
        stack = []
        minLeftIndex = [-1 for i in range(0,len(arr))]
        minEqualRightIndex = [len(arr) for i in range(0,len(arr))]
        for index in range(0, len(arr)):
            elem = arr[index]
            while len(stack)!=0 and arr[stack[-1]]>=elem:
                nextMin = stack.pop()
                minEqualRightIndex[nextMin] = index
            stack.append(index)

        stack = []
        for index in range(len(arr)-1, -1, -1):
            elem = arr[index]
            while len(stack)!=0 and arr[stack[-1]]>elem:
                nextMin = stack.pop()
                minLeftIndex[nextMin] = index
            stack.append(index)

        summation = 0
        for index in range(0, len(arr)):
            summation += arr[index]*(index-minLeftIndex[index])*(minEqualRightIndex[index
            if summation>=1000000007:
                summation -= 1000000007
        return summation
```

2485 Finding the Number of Visible Mountains (link)

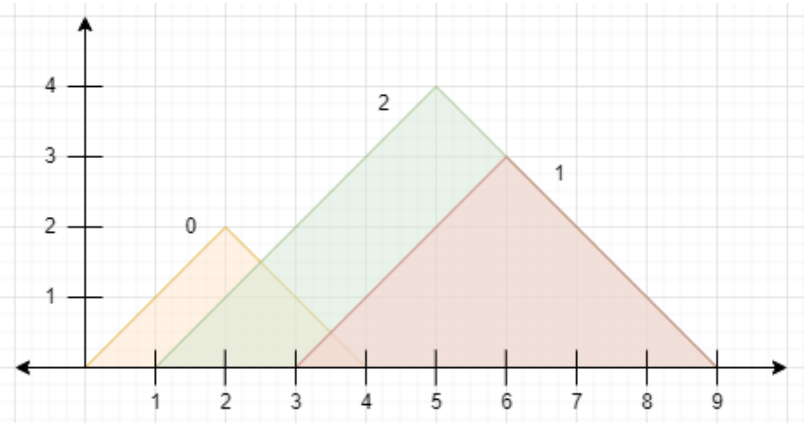
Description

You are given a **0-indexed** 2D integer array `peaks` where `peaks[i] = [x_i, y_i]` states that mountain `i` has a peak at coordinates `(x_i, y_i)`. A mountain can be described as a right-angled isosceles triangle, with its base along the `x`-axis and a right angle at its peak. More formally, the **gradients** of ascending and descending the mountain are `1` and `-1` respectively.

A mountain is considered **visible** if its peak does not lie within another mountain (including the border of other mountains).

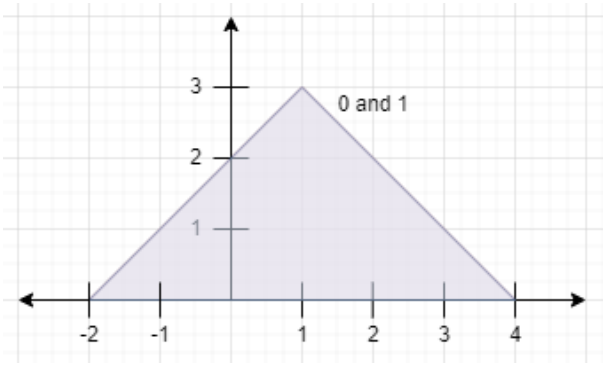
Return *the number of visible mountains*.

Example 1:




```
Input: peaks = [[2,2],[6,3],[5,4]]
Output: 2
Explanation: The diagram above shows the mountains.
- Mountain 0 is visible since its peak does not lie within another mountain or its sides
- Mountain 1 is not visible since its peak lies within the side of mountain 2.
- Mountain 2 is visible since its peak does not lie within another mountain or its sides
There are 2 mountains that are visible.
```

Example 2:




```
Input: peaks = [[1,3],[1,3]]
Output: 0
Explanation: The diagram above shows the mountains (they completely overlap).
Both mountains are not visible since their peaks lie within each other.
```

Constraints:

- 1 <= peaks.length <= 10^5
- peaks[i].length == 2
- 1 <= x_i, y_i <= 10^5

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
from operator import itemgetter

class Solution:
    def lastPeakInCurrent(self, lastPeak, currentPeak):
        y = currentPeak[1] - (currentPeak[0]-lastPeak[0])
        if lastPeak[1]>y:
            return False
        else:
            return True

    def visibleMountainsInternal(self, peaks):
        stack = [peaks[0]]
        for peakIndex in range(1, len(peaks)):
            lastElem = stack[-1]
            currElem = peaks[peakIndex]
            # while len(stack)>1 and self.lastPeakInCurrent(lastElem, currElem):
            #     stack.pop()
            #     lastElem = stack[-1]
            y = lastElem[1]-(currElem[0]-lastElem[0])
            if currElem[1]>y:
                stack.append(currElem)
        return stack
    def visibleMountainsReverse(self, peaks):
        stack = [peaks[-1]]
        for peakIndex in range(len(peaks)-2, -1, -1):
            lastElem = stack[-1]
            currElem = peaks[peakIndex]
            y = lastElem[1]-(lastElem[0]-currElem[0])
            if currElem[1]>y:
                stack.append(currElem)
        return stack

    def visibleMountains(self, peaks: List[List[int]]) -> int:
        # Finding distinct peaks
        distinctPeaks = {}
        for peak in peaks:
            peakStr = str(peak[0])+"_"+str(peak[1])
            if peakStr not in distinctPeaks:
                distinctPeaks[peakStr] = 0
            distinctPeaks[peakStr] += 1

        peaks = [[int(peakStr.split("_")[0]), int(peakStr.split("_")[1])] for peakStr in
        # Sorting peaks by x
        peaks = sorted(peaks, key=itemgetter(0))
        if len(peaks)==0:
            return 0
        leftRightPeaks = self.visibleMountainsInternal(peaks)
        rightLeftPeaks = self.visibleMountainsReverse(leftRightPeaks)

        outPeaks = []
        for peak in rightLeftPeaks:
            peakStr = str(peak[0])+"_"+str(peak[1])
            if distinctPeaks[peakStr]==1:
                outPeaks.append(peak)
```

873 Guess the Word (link)

Description

You are given an array of unique strings `words` where `words[i]` is six letters long. One word of `words` was chosen as a secret word.

You are also given the helper object `Master`. You may call `Master.guess(word)` where `word` is a six-letter-long string, and it must be from `words`. `Master.guess(word)` returns:

- `-1` if `word` is not from `words`, or
- an integer representing the number of exact matches (value and position) of your guess to the secret word.

There is a parameter `allowedGuesses` for each test case where `allowedGuesses` is the maximum number of times you can call `Master.guess(word)`.

For each test case, you should call `Master.guess` with the secret word without exceeding the maximum number of allowed guesses. You will get:

- **`"Either you took too many guesses, or you did not find the secret word."`** if you called `Master.guess` more than `allowedGuesses` times or if you did not call `Master.guess` with the secret word, or
- **`"You guessed the secret word correctly."`** if you called `Master.guess` with the secret word with the number of calls to `Master.guess` less than or equal to `allowedGuesses`.

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the bruteforce method).

Example 1:

```
Input: secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"], allowedGuesses =
Output: You guessed the secret word correctly.
Explanation:
master.guess("aaaaaa") returns -1, because "aaaaaa" is not in wordlist.
master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6 matches.
master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.
master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.
master.guess("abcczz") returns 4, because "abcczz" has 4 matches.
We made 5 calls to master.guess, and one of them was the secret, so we pass the test case
```

Example 2:

```
Input: secret = "hamada", words = ["hamada","khaled"], allowedGuesses = 10
Output: You guessed the secret word correctly.
Explanation: Since there are two words, you can guess both.
```

Constraints:

- `1 <= words.length <= 100`
- `words[i].length == 6`
- `words[i]` consist of lowercase English letters.
- All the strings of `wordlist` are **unique**.
- `secret` exists in `words`.
- `10 <= allowedGuesses <= 30`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
# """
# This is Master's API interface.
# You should not implement it, or speculate about its implementation
# """
# class Master:
#     def guess(self, word: str) -> int:

class Solution:
    def findIntersectingWord(self, words, word, matchCount):
        newList = []
        for index in range(0, len(words)):
            if word==words[index]:
                continue
            count = 0
            for charIndex in range(0, 6):
                if words[index][charIndex]==word[charIndex]:
                    count += 1
            if count>=matchCount:
                newList += [words[index]]
        return newList

    def mostCommonPattern(self, words):
        patterns = {}
        for word in words:
            for i in range(0, 6):
                pattern = str(i)+word[i]
                if pattern in patterns:
                    patterns[pattern][0] += 1
                    patterns[pattern][1] += [word]
                else:
                    patterns[pattern] = [1, [word]]
        maxCount = 0
        maxPatternWords = []
        for pattern in patterns:
            if patterns[pattern][0]>maxCount:
                maxCount = patterns[pattern][0]
                maxPatternWords = patterns[pattern][1]
        return maxPatternWords

    def findSecretWord(self, words: List[str], master: 'Master') -> None:
        wordsCopy = [word for word in words]
        count = 1
        while len(wordsCopy)>0:
            maxPatternWords = self.mostCommonPattern(wordsCopy)
            word = maxPatternWords.pop()
            wordsCopy.remove(word)
            matchCount = master.guess(word)
            if matchCount==6:
                break
            elif matchCount==0:
                for removeWord in maxPatternWords:
                    if removeWord in wordsCopy:
                        wordsCopy.remove(removeWord)
            else:
```

```
                wordsCopy = self.findIntersectingWord(wordsCopy, word, matchCount)
            count += 1
```

1274 Number of Days Between Two Dates (link)

Description

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is `YYYY-MM-DD` as shown in the examples.

Example 1:

```
Input: date1 = "2019-06-29", date2 = "2019-06-30"
Output: 1
```

Example 2:

```
Input: date1 = "2020-01-15", date2 = "2019-12-31"
Output: 15
```

Constraints:

- The given dates are valid dates between the years `1971` and `2100`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def isLeep(self, year):
        if (year%100)==0:
            if (year%400)==0:
                return True
            else:
                return False
        elif (year%4)==0:
            return True
        else:
            return False

    def daysBetweenDates(self, date1: str, date2: str) -> int:
        daysNormal = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
        daysLeep = [31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
        date1Split = [int(number) for number in date1.split("-")]
        date2Split = [int(number) for number in date2.split("-")]
        startDate = date1Split
        endDate = date2Split
        if date1Split[0]>date2Split[0]:
            startDate = date2Split
            endDate = date1Split
        elif date1Split[0]==date2Split[0] and date1Split[1]>date2Split[1]:
            startDate = date2Split
            endDate = date1Split
        elif (date1Split[1]==date2Split[1] and date1Split[0]==date2Split[0]) and date1Sp
            startDate = date2Split
            endDate = date1Split

        daysCount = 0
        for year in range(startDate[0], endDate[0]+1):
            if self.isLeep(year):
                daysCount += 366
            else:
                daysCount += 365

        for month in range(0, startDate[1]):
            if not self.isLeep(startDate[0]):
                daysCount -= daysNormal[month]
            else:
                daysCount -= daysLeep[month]

        for month in range(endDate[1]-1, 12):
            if not self.isLeep(endDate[0]):
                daysCount -= daysNormal[month]
            else:
                daysCount -= daysLeep[month]

        if not self.isLeep(startDate[0]):
            daysCount += daysNormal[startDate[1]-1]-startDate[2]
        else:
            daysCount += daysLeep[startDate[1]-1]-startDate[2]
```

```
        daysCount += endDate[2]
        return daysCount
```

63 Unique Paths II (link)

Description

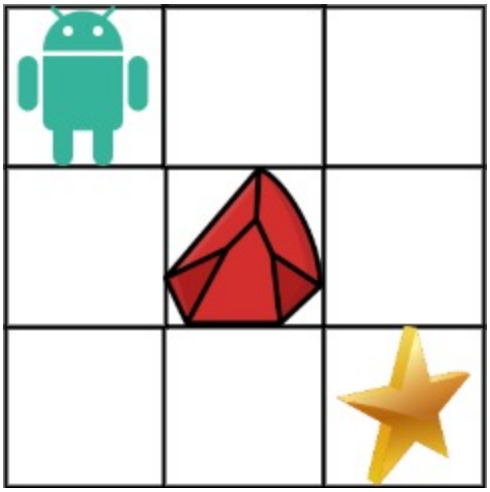
You are given an `m x n` integer array `grid`. There is a robot initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as `1` or `0` respectively in `grid`. A path that the robot takes cannot include **any** square that is an obstacle.

Return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

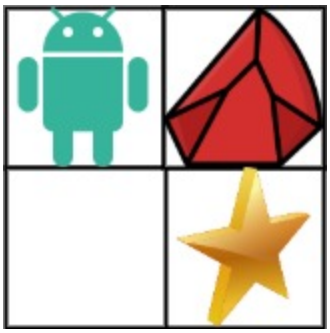
The testcases are generated so that the answer will be less than or equal to `2 * 10^9`.

Example 1:



```
Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Output: 2
Explanation: There is one obstacle in the middle of the 3x3 grid above.
There are two ways to reach the bottom-right corner:
1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right
```

Example 2:



```
Input: obstacleGrid = [[0,1],[0,0]]
Output: 1
```

Constraints:

- `m == obstacleGrid.length`
- `n == obstacleGrid[i].length`
- `1 <= m, n <= 100`
- `obstacleGrid[i][j]` is `0` or `1`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def __init__(self):
        self.calculatedValues = []
    def topDown(self, grid, currentRow, currentCol):
        if grid[currentRow][currentCol]==1:
            return 0
        if currentRow==0 and currentCol==0:
            self.calculatedValues[currentRow][currentCol] = 1
            return 1
        if self.calculatedValues[currentRow][currentCol]:
            return self.calculatedValues[currentRow][currentCol]
        count = 0
        if currentRow>0:
            upRow = self.topDown(grid, currentRow-1, currentCol)
            count += upRow
        if currentCol>0:
            leftCol = self.topDown(grid, currentRow, currentCol-1)
            count += leftCol
        self.calculatedValues[currentRow][currentCol] = count
        #print(self.calculatedValues)
        return count

    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        self.calculatedValues = [[None for i in obstacleGrid[0]] for j in obstacleGrid]
        return self.topDown(obstacleGrid, len(obstacleGrid)-1, len(obstacleGrid[0])-1)
```

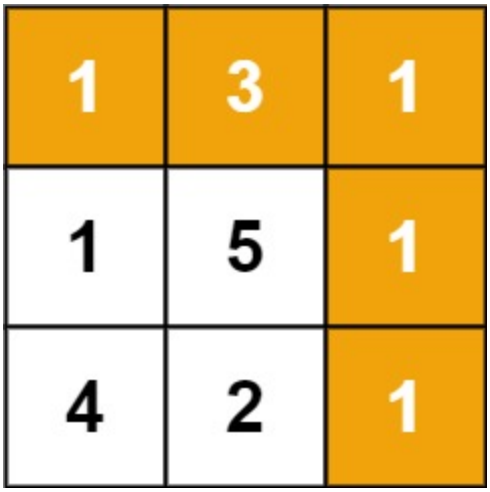
64 Minimum Path Sum (link)

Description

Given a `m x n grid` filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:




```
Input: grid = [[1,3,1],[1,5,1],[4,2,1]]
Output: 7
Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.
```

Example 2:

```
Input: grid = [[1,2,3],[4,5,6]]
Output: 12
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 200`
- `0 <= grid[i][j] <= 200`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def __init__(self):
        self.culatedValues = []

    def calualateMin(self, grid, currentRow, currentIndex):
        if currentRow==0 and currentIndex==0:
            self.culatedValues[currentRow][currentIndex] = grid[currentRow][currentIndex]
            return grid[currentRow][currentIndex]
        if self.culatedValues[currentRow][currentIndex]:
            return self.culatedValues[currentRow][currentIndex]
        minVal = sys.maxsize
        if currentRow>0:
            minVal = min(minVal, self.calualateMin(grid, currentRow-1, currentIndex))
        if currentIndex>0:
            minVal = min(self.calualateMin(grid, currentRow, currentIndex-1),minVal)
        minVal += grid[currentRow][currentIndex]
        self.culatedValues[currentRow][currentIndex] = minVal
        return minVal
    def minPathSum(self, grid: List[List[int]]) -> int:
        self.culatedValues = [[None for i in grid[0]] for j in grid]
        minVal = self.calualateMin(grid, len(grid)-1, len(grid[0])-1)
        return minVal
```

139 Word Break (link)

Description

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false
```

Constraints:

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        length = len(s)
        maxLengthWord = 0
        words = {}
        for word in wordDict:
            words[word] = 1
            if len(word)>maxLengthWord:
                maxLengthWord = len(word)
        staringStart = [False]*length
        for index in range(1, length+1):
            for startIndex in range(max(0, index-maxLengthWord), index):
                if startIndex==0 or staringStart[startIndex-1]:
                    currStr = s[startIndex:index]
                    if currStr in words:
                        staringStart[index-1] = True
        #print(staringStart)
        return staringStart[-1]
```

70 Climbing Stairs (link)

Description

You are climbing a staircase. It takes `n` steps to reach the top.

Each time you can either climb `1` or `2` steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- `1 <= n <= 45`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        if n==0:
            return 0
        elif n==1:
            return 1
        calculatesVaues = [-1]*n
        calculatesVaues[-1] = 1
        calculatesVaues[-2] = 2
        for i in range(n-3, -1, -1):
            calculatesVaues[i] =  calculatesVaues[i+1] +calculatesVaues[i+2]
        return calculatesVaues[0]
```

2487 Optimal Partition of String (link)

Description

Given a string `s`, partition the string into one or more **substrings** such that the characters in each substring are **unique**. That is, no letter appears in a single substring more than **once**.

Return *the **minimum** number of substrings in such a partition.*

Note that each character should belong to exactly one substring in a partition.

Example 1:

```
Input: s = "abacaba"
Output: 4
Explanation:
Two possible partitions are ("a","ba","cab","a") and ("ab","a","ca","ba").
It can be shown that 4 is the minimum number of substrings needed.
```

Example 2:

```
Input: s = "ssssss"
Output: 6
Explanation:
The only valid partition is ("s","s","s","s","s","s").
```

Constraints:

- `1 <= s.length <= 10^5`
- `s` consists of only English lowercase letters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def partitionString(self, s: str) -> int:
        charsFound = {}
        countFound = 0
        for char in s:
            if char in charsFound:
                countFound += 1
                charsFound = {}
                charsFound[char] = 1
            else:
                charsFound[char] = 1

        if len(charsFound)>0:
            countFound += 1
        return countFound
```

1585 The kth Factor of n (link)

Description

You are given two positive integers `n` and `k`. A factor of an integer `n` is defined as an integer `i` where `n % i == 0`.

Consider a list of all factors of `n` sorted in **ascending order**, return *the* k^{th} *factor* in this list or return `-1` if `n` has less than `k` factors.

Example 1:

```
Input: n = 12, k = 3
Output: 3
Explanation: Factors list is [1, 2, 3, 4, 6, 12], the 3rd factor is 3.
```

Example 2:

```
Input: n = 7, k = 2
Output: 7
Explanation: Factors list is [1, 7], the 2nd factor is 7.
```

Example 3:

```
Input: n = 4, k = 4
Output: -1
Explanation: Factors list is [1, 2, 4], there is only 3 factors. We should return -1.
```

Constraints:

- `1 <= k <= n <= 1000`

Follow up:

Could you solve this problem in less than O(n) complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def kthFactor(self, n: int, k: int) -> int:
        count = 0
        for i in range(1,n+1):
            if (n%i)==0:
                count += 1
                if count==k:
                    return i
        return -1
```

295 Find Median from Data Stream (link)

Description

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for `arr = [2,3,4]`, the median is `3`.
- For example, for `arr = [2,3]`, the median is `(2 + 3) / 2 = 2.5`.

Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

```
Input
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output
[null, null, null, 1.5, null, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Constraints:

- -10^5 `<= num <=` 10^5
- There will be at least one element in the data structure before calling `findMedian`.
- At most `5 *` 10^4 calls will be made to `addNum` and `findMedian`.

Follow up:

- If all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?
- If `99%` of all integer numbers from the stream are in the range `[0, 100]`, how would you optimize your solution?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
import heapq

class MedianFinder:

    def __init__(self):
        self.numbersFirstHalf = []
        self.numbersSecondHalf = []

    def addNum(self, num: int) -> None:
        if len(self.numbersFirstHalf)==0:
            heapq.heappush(self.numbersFirstHalf, num*-1)
        elif len(self.numbersSecondHalf)==0:
            firstListNum = heapq.heappop(self.numbersFirstHalf)*-1
            if firstListNum<num:
                heapq.heappush(self.numbersSecondHalf, num)
                heapq.heappush(self.numbersFirstHalf, firstListNum*-1)
            else:
                heapq.heappush(self.numbersSecondHalf, firstListNum)
                heapq.heappush(self.numbersFirstHalf, num*-1)
        else:
            firstListNum = heapq.heappop(self.numbersFirstHalf)*-1
            secondListNum = heapq.heappop(self.numbersSecondHalf)
            if len(self.numbersFirstHalf) <= len(self.numbersSecondHalf):
                heapq.heappush(self.numbersFirstHalf, firstListNum*-1)
                if num>secondListNum:
                    heapq.heappush(self.numbersSecondHalf, num)
                    heapq.heappush(self.numbersFirstHalf, secondListNum*-1)
                else:
                    heapq.heappush(self.numbersFirstHalf, num*-1)
                    heapq.heappush(self.numbersSecondHalf, secondListNum)
            else:
                heapq.heappush(self.numbersSecondHalf, secondListNum)
                if num>firstListNum:
                    heapq.heappush(self.numbersSecondHalf, num)
                    heapq.heappush(self.numbersFirstHalf, firstListNum*-1)
                else:
                    heapq.heappush(self.numbersFirstHalf, num*-1)
                    heapq.heappush(self.numbersSecondHalf, firstListNum)

    def findMedian(self) -> float:
        firstListNum = heapq.heappop(self.numbersFirstHalf)*-1
        heapq.heappush(self.numbersFirstHalf, firstListNum*-1)
        if len(self.numbersFirstHalf)>len(self.numbersSecondHalf):
            return round(float(firstListNum),5)
        else:
            secondListNum = heapq.heappop(self.numbersSecondHalf)
            heapq.heappush(self.numbersSecondHalf, secondListNum)
            print(round(float(firstListNum+secondListNum)/2, 5))
            return round(float(firstListNum+secondListNum)/2, 5)


# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
```

```
# obj.addNum(num)
# param_2 = obj.findMedian()
```

373 Find K Pairs with Smallest Sums (link)

Description

You are given two integer arrays `nums1` and `nums2` sorted in **non-decreasing order** and an integer `k`.

Define a pair `(u, v)` which consists of one element from the first array and one element from the second array.

Return *the* `k` *pairs* $(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k)$ *with the smallest sums*.

Example 1:

```
Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Output: [[1,2],[1,4],[1,6]]
Explanation: The first 3 pairs are returned from the sequence: [1,2],[1,4],[1,6],[7,2],[
```

Example 2:

```
Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Output: [[1,1],[1,1]]
Explanation: The first 2 pairs are returned from the sequence: [1,1],[1,1],[1,2],[2,1],[
```

Constraints:

- `1 <= nums1.length, nums2.length <=` 10^5
- `-`10^9 `<= nums1[i], nums2[i] <=` 10^9
- `nums1` and `nums2` both are sorted in **non-decreasing order**.
- `1 <= k <=` 10^4
- `k <= nums1.length * nums2.length`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
import heapq
class Solution:
    def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int
        heap = []
        output = []
        for index in range(0, len(nums1)):
            heapq.heappush(heap,(nums1[index]+nums2[0], [index, 0]))

        currentNums2Index = 0
        while k>0:
            elem = heapq.heappop(heap)
            output += [[nums1[elem[1][0]], nums2[elem[1][1]]]]
            if elem[1][1]<(len(nums2)-1):
                heapq.heappush(heap,(nums1[elem[1][0]]+nums2[elem[1][1]+1], [elem[1][0],
            k -= 1
        return output
```

274 H-Index (link)

Description

Given an array of integers `citations` where `citations[i]` is the number of citations a researcher received for their i^{th} paper, return *the researcher's h-index*.

According to the definition of h-index on Wikipedia: The h-index is defined as the maximum value of `h` such that the given researcher has published at least `h` papers that have each been cited at least `h` times.

Example 1:

```
Input: citations = [3,0,6,1,5]
Output: 3
Explanation: [3,0,6,1,5] means the researcher has 5 papers in total and each of them had
Since the researcher has 3 papers with at least 3 citations each and the remaining two wi
```

Example 2:

```
Input: citations = [1,3,1]
Output: 1
```

Constraints:

- `n == citations.length`
- `1 <= n <= 5000`
- `0 <= citations[i] <= 1000`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def hIndex(self, citations: List[int]) -> int:
        citations.sort()
        count = 0
        for i in range(len(citations)-1, -1, -1):
            count += 1
            if citations[i]<count:
                count -= 1
                break
        return count
```

45 Jump Game II (link)

Description

You are given a **0-indexed** array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- `0 <= j <= nums[i]` and
- `i + j < n`

Return *the minimum number of jumps to reach index* `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: 2
Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from i
```

Example 2:

```
Input: nums = [2,3,0,1,4]
Output: 2
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 1000`
- It's guaranteed that you can reach `nums[n - 1]`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def jump(self, nums: List[int]) -> int:
        currentIndex = len(nums)-1
        minJumpsArray = [currentIndex+1]*(currentIndex+1)
        minJumpsArray[currentIndex] = 0
        while currentIndex>0:
            currJumpIndex = currentIndex-1
            while currJumpIndex<(nums[currentIndex-1]+currentIndex-1) and currJumpIndex<
                currJumpIndex += 1
                minJumpsArray[currentIndex-1] = min(minJumpsArray[currentIndex-1], minJu
            currentIndex -= 1

        return minJumpsArray[0]
```

55 Jump Game (link)

Description

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` *if you can reach the last index, or* `false` *otherwise*.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

Example 2:

```
Input: nums = [3,2,1,0,4]
Output: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is
```

Constraints:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 10^5`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        startIndex = len(nums)-1
        while startIndex>0:
            pastIndex = startIndex
            found = False
            while pastIndex>0:
                pastIndex -= 1
                if nums[pastIndex]>=(startIndex-pastIndex):
                    startIndex = pastIndex
                    found = True
                    break
            if not found:
                return False
        return True
```

122 Best Time to Buy and Sell Stock II (link)

Description

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

On each day, you may decide to buy and/or sell the stock. You can only hold **at most one** share of the stock at any time. However, you can sell and buy the stock multiple times on the **same day**, ensuring you never hold than one share of the stock.

Find and return *the **maximum** profit you can achieve*.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.
```

Example 3:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achi
```

Constraints:

- `1 <= prices.length <= 3 * 10^4`
- `0 <= prices[i] <= 10^4`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        totalProfit = 0
        length = len(prices)
        for i in range(1, length):
            currentProfit = prices[i]-prices[i-1]
            if currentProfit>0:
                totalProfit += currentProfit
        return totalProfit
```

121 Best Time to Buy and Sell Stock (link)

Description

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before
```

Example 2:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.
```

Constraints:

- `1 <= prices.length <= 10^5`
- `0 <= prices[i] <= 10^4`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        maximumProfit = 0
        minBuyIndex = 0
        minBuyPrice = prices[0]
        length = len(prices)
        for index in range(0, length):
            currentProfit = prices[index]-prices[minBuyIndex]
            if currentProfit>maximumProfit:
                maximumProfit = currentProfit
            if prices[index]<minBuyPrice:
                minBuyPrice = prices[index]
                minBuyIndex = index

        return maximumProfit
```

189 Rotate Array (link)

Description

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- $1 <= nums.length <= 10^5$
- $-2^{31} <= nums[i] <= 2^{31} - 1$
- $0 <= k <= 10^5$

Follow up:

- Try to come up with as many solutions as you can. There are at least **three** different ways to solve this problem.
- Could you do it in-place with `O(1)` extra space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def revert(self, array, startIndex, endIndex):
        while startIndex<endIndex:
            temp = array[startIndex]
            array[startIndex] = array[endIndex]
            array[endIndex] = temp
            startIndex += 1
            endIndex -= 1

    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        length = len(nums)
        k = k%length
        if k>0:
            self.revert(nums, 0, length-1)
            self.revert(nums, 0,k-1)
            self.revert(nums, k,length-1)
```

169 Majority Element (link)

Description

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-10^9 <= nums[i] <= 10^9`

Follow-up: Could you solve the problem in linear time and in `O(1)` space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        count = 0
        value = -1
        for index in range(0,len(nums)):
            if count==0:
                value = nums[index]
            if value==nums[index]:
                count += 1
            else:
                count -= 1
        return value
```

80 Remove Duplicates from Sorted Array II (link)

Description

Given an integer array `nums` sorted in **non-decreasing order**, remove some duplicates **in-place** such that each unique element appears **at most twice**. The **relative order** of the elements should be kept the **same**.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the **first part** of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` *after placing the final result in the first* `k` *slots of* `nums`.

Do **not** allocate extra space for another array. You must do this by **modifying the input array in-place** with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of nums beir
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_,_]
Explanation: Your function should return k = 7, with the first seven elements of nums be
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        length = len(nums)
        updateIndex = 2
        for readIndex in range(2,length):
            if nums[readIndex] != nums[updateIndex-1] or nums[readIndex] != nums[updateIn
                nums[updateIndex] = nums[readIndex]
                updateIndex += 1
        return updateIndex
```

26 Remove Duplicates from Sorted Array (link)

Description

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**. Then return *the number of unique elements in* `nums`.

Consider the number of unique elements of `nums` to be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the unique elements in the order they were present in `nums` initially. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,2]
Output: 2, nums = [1,2,_]
Explanation: Your function should return k = 2, with the first two elements of nums being
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums beir
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-100 <= nums[i] <= 100`
- `nums` is sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        length = len(nums)
        updateIndex = 1
        for readIndex in range(1,length):
            if nums[readIndex] != nums[updateIndex-1]:
                nums[updateIndex] = nums[readIndex]
                updateIndex += 1
        return updateIndex
```

88 Merge Sorted Array (link)

Description

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

Example 1:

```
Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.
```

Example 2:

```
Input: nums1 = [1], m = 1, nums2 = [], n = 0
Output: [1]
Explanation: The arrays we are merging are [1] and [].
The result of the merge is [1].
```

Example 3:

```
Input: nums1 = [0], m = 0, nums2 = [1], n = 1
Output: [1]
Explanation: The arrays we are merging are [] and [1].
The result of the merge is [1].
Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure th
```

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-10^9 <= nums1[i], nums2[j] <= 10^9`

Follow up: Can you come up with an algorithm that runs in `O(m + n)` time?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """
        nums1Index = m-1
        nums2Index = n-1
        storeIndex = m+n-1
        while nums1Index>=0 and nums2Index>=0:
            if nums1[nums1Index] > nums2[nums2Index]:
                nums1[storeIndex] = nums1[nums1Index]
                nums1Index -= 1
            else:
                nums1[storeIndex] = nums2[nums2Index]
                nums2Index -= 1
            storeIndex -= 1

        while nums2Index>=0:
            nums1[storeIndex] = nums2[nums2Index]
            nums2Index -= 1
            storeIndex -= 1
```

27 Remove Element (link)

Description

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in* `nums` *which are not equal to* `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                            // It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,2,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

```
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums cont
```

```
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- 0 <= nums.length <= 100
- 0 <= nums[i] <= 50
- 0 <= val <= 100

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```python
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        length = len(nums)
        count = 0
        index = length-1
        while index>=0:
            if nums[index]==val:
                temp = nums[length-count-1]
                nums[length-count-1] = nums[index]
                nums[index] = temp
                count += 1
            index -= 1

        return length-count
```